

Preparazione Esame — Astralis

Riferimento completo per l'esame finale. Script apertura, traccia vs realizzato, Q&A, definizioni, live coding.

1. Script di Apertura (5 min)

Ordine consigliato per dimostrare il progetto:

- 1. **Login** admin@astralis.it / password tema dark Breeze
- 2. **Dashboard** grafici Chart.js (donut categorie, barre tipi, barre missioni)
- 3. **Corpi Celesti** lista con filtri, ricerca, paginazione apri "Terra" metriche, galleria lightbox, curiosità, timeline missioni, corpi simili
- 4. **Categorie** badge colorati, conteggio corpi associati
- 5. **Missioni** timeline orizzontale, badge stato (Completata/In corso/Pianificata)
- 6. **NASA Import** bottone "Importa da NASA" immagini reali
- 7. **Guest React** localhost:5175 sistema solare animato griglia corpi dettaglio con lightbox comparatore

2. Traccia Realizzato

Parte 1 — Backoffice Laravel

#	Requisito traccia	Realizzato	Dettagli
1	Backoffice con autenticazione Breeze		Breeze Blade puro (Inertia rimosso). Route protette da middleware auth . Login, register, profilo, reset password — tema dark #0A0A1A
2	CRUD entità principale		CorpoCeleste: 7 metodi controller, form 6 sezioni, 18 campi
3	Relazioni 1-N o N-N		5 entità, 3 tipi: BelongsTo/HasMany (1-N), BelongsToMany (N-N con pivot corpo_celeste_missione)
4	CRUD entità secondarie		Categoria, Missione, Curiosità, Galleria — CRUD completo ciascuna

5	Upload media		Missioni (logo 300px), Galleria (1200px) con ImageUploadService + Intervention Image v4 <code>scaleDown()</code>
6	Template Blade		Layout master <code>app.blade.php</code> con sidebar navigazione, Alpine.js (npm)

Parte 2 — Guest React

#	Requisito traccia	Realizzato	Dettagli
7	SPA React guest		React 18 standalone, Vite, 5 pagine (Home, Lista, Dettaglio, Comparatore, 404)
8	Lista elementi		<code>CorpiLista.jsx</code> : griglia card, filtri (categoria, tipo, ricerca), paginazione “Carica altri”
9	Dettaglio elemento		<code>CorpoDettaglio.jsx</code> : metriche, galleria lightbox, curiosità, timeline missioni, simili
10	Info correlate		Badge categoria, lightbox immagini NASA, timeline missioni, corpi simili
11	API REST		10 endpoint JSON pubblici in <code>routes/api.php</code> . API Resources, eager loading, filtri query
12	Test Postman		Endpoint pubblici, JSON response, filtri via query params

Extra Wow Factor

Extra	Descrizione	Dove
NASA API Integration	Import automatico immagini reali, fallback apostrofi, auto-import su created via Observer Job	Fase 8-9
Sistema Solare Animato	8 pianeti orbitanti con <code>requestAnimationFrame</code> , velocità differenziate	Fase 6
Lightbox Galleria	Schermo intero con swipe mobile, slideshow	Fase 5
Comparatore Pianeti	Confronto 2 corpi su 7 metriche, pre-fill via URL params	Fase 5
Timeline Missioni	Scrolling orizzontale con badge stato colorato	Fase 5
Dashboard Chart.js	3 grafici donut/barre con tema dark	Fase 14

CLI Commands	<code>astralis:fetch-nasa</code> e <code>astralis:gallery</code> per manutenzione	Fase 8-11
Error Boundary	Fallback UI globale per crash React	Fase 15
SEO	<code>document.title</code> dinamico su 5 pagine React	Fase 15
381 Test	271 PHPUnit + 110 Vitest, <code>Http::fake()</code> , <code>observer skip</code>	Fase 13+
WordMapService	Traduzione italiano → inglese per ricerca NASA (~70 termini)	Fase 9
Responsive	Navbar mobile, SolarSystem responsive scaling, griglia adattiva	Ongoing

3. Postman — Esempi Pratici

Tutti gli endpoint sono **pubblici** (nessuna autenticazione). Puntare a `http://localhost:8000` .

Come testare su Postman

1. Avvia il backend: `php artisan serve` (porta 8000)
2. Crea una request GET in Postman
3. Inserisci l'URL: `http://localhost:8000/api/corpi-celesti`
4. Nessun token o autenticazione necessaria
5. Parametri query opzionali: `?categoria=pianeta&search=terra&in_evidenza=1&per_page=5`

Esempio 1 — Homepage (stats + in evidenza)

Step 1: Statistiche generali

GET `http://localhost:8000/api/dashboard/stats`

Response 200:

```
{
  "totale_corpi_celesti": 18,
  "totale_categorie": 8,
  "totale_missioni": 10,
  "corpi_in_evidenza": 9,
  "ultimi_corpi": [...],
  "missioni_per_stato": {
    "total": 10,
    "completate": 4,
    "in_corso": 4,
```

```
"pianificate": 2
}
}
```

Step 2: Corpi in evidenza

GET http://localhost:8000/api/corpi-celesti?in_evidenza=1&per_page=6

Esempio 2 — Dettaglio Terra

GET <http://localhost:8000/api/corpi-celesti/terra>

Response 200 (estratto):

```
{
  "data": {
    "id": 3,
    "nome": "Terra",
    "slug": "terra",
    "categoria": {
      "id": 1,
      "nome": "Pianeta",
      "slug": "pianeta",
      "colore": "#22D3EE"
    },
    "immagine_url": "https://images-assets.nasa.gov/image/PIA18033/PIA18033~medium.jpg",
    "tipo": "Pianeta roccioso",
    "massa_kg": "5.972e24",
    "diametro_km": "12756",
    "gravita": "9.81",
    "temperatura": "15",
    "galleria": [...],
    "curiosita": [...],
    "missioni": [
      {
        "nome": "Apollo 11",
        "pivot": {
          "tipo_esplorazione": "missione con equipaggio",
          "anno_arrivo": 1969
        }
      }
    ]
  }
}
```

```
}  
}
```

Nota: `nome` contiene il nome italiano. `nome_en` (opzionale) contiene l'inglese.

Esempio 3 — Filtri multipli

```
GET http://localhost:8000/api/missioni?agenzia=NASA&stato=in corso
```

Esempio 4 — Corpi simili

```
GET http://localhost:8000/api/corpi-celesti/terra/simili
```

Restituisce max 4 corpi della stessa categoria.

4. Stack Tecnologico

Livello	Tecnologia	Versione	Perché
Backend	Laravel	13.8	Richiesto dalla traccia. Eloquent ORM, migrazioni, artisan CLI
Database	MySQL	8.x	Richiesto. Porta 3307
Auth	Laravel Breeze	—	Richiesto. Configurato con Blade (non Inertia)
Frontend guest	React	18.2	Richiesto. SPA standalone con Vite
Frontend admin	Blade + Alpine.js	3.15	Richiesto per admin. Alpine.js bundled via Vite per interattività
CSS	Tailwind CSS	4.3	Utility-first, tema dark custom
Icone	lucide-react	1.23	Icone categoria, navigazione, azioni
Lightbox	yet-another-react-lightbox	—	Galleria immagini a schermo intero
Immagini	Intervention Image	4	Upload con <code>scaleDown</code> via <code>ImageUploadService</code>

Slug	spatie/laravel-sluggable	—	Slug automatici su 3 modelli
Grafici	Chart.js	4.4	Dashboard admin (donut, barre), bundled via Vite
API esterne	NASA Image API	—	Import immagini reali, auto-suggest
Test PHP	PHPUnit	12.5	271 test, 615 assertion
Test JS	Vitest + Testing Library	4.1	110 test, environment jsdom

5. Architettura & Mappa File

Backend (Laravel)

File	Dove
Route admin	routes/web.php gruppo admin
Route API	routes/api.php 10 endpoint
Route auth	routes/auth.php Breeze
Controllers admin	app/Http/Controllers/Admin/ (8 file)
Controllers API	app/Http/Controllers/Api/ (6 file)
Models	app/Models/ (6 file: 5 entità + User)
Migrations	database/migrations/ (21 file)
Services	app/Services/ (NasalImageService, WordMapService, ImageUploadService)
Policies	app/Policies/ (5 file)
Gate admin	app/Providers/AuthServiceProvider.php
Observer	app/Observers/CorpoCelesteObserver.php
Job	app/Jobs/ImportNasalImage.php
FormRequest	app/Http/Requests/ (13 file)
Config admin	config/admin.php

Layout admin	resources/views/admin/layouts/app.blade.php
Sidebar	resources/views/admin/partials/_sidebar-nav.blade.php
CSS entry	resources/css/app.css — Tailwind entry + [x-cloak]

Frontend (React)

File	Dove
Entry point	resources/js/guest/main.jsx
Routing	resources/js/guest/App.jsx
Pages	resources/js/guest/pages/ (5 pagine)
Components	resources/js/guest/components/
API client	resources/js/guest/apiClient.js
Custom hooks	resources/js/guest/hooks/ (useFetch, useDebounce)
Admin JS	resources/js/admin.js — Alpine.js entry (npm)
Admin charts	resources/js/admin-charts.js — Chart.js entry (npm)
Vite config	vite.config.js (proxy /api localhost:8000)

6. Q&A — Risposte alle Domande (20+)

Architettura

Q: Dove sono le API?

routes/api.php . 10 endpoint GET. Controller in app/Http/Controllers/Api/ . API Resources per JSON controllato.

Q: Dove sono le CRUD?

app/Http/Controllers/Admin/ . Ogni controller ha 7 metodi: index, create, store, show, edit, update, destroy. Route resource in routes/web.php .

Q: Come funziona il routing?

Due mondi: admin (/admin/*) gestito da Laravel + Blade; guest (/*) gestito da React SPA. Il catch-all {any} in web.php passa tutto a guest.blade.php .

Q: Qual è la differenza tra admin e guest?

Admin: Blade + Alpine.js, autenticazione Breeze, CRUD. Guest: React SPA standalone, API REST, animazioni. Comunicano solo via API.

Q: Perché avete separato frontend guest e backoffice admin?

Per responsabilità e competenze diverse. Il guest è una SPA standalone che non richiede autenticazione e beneficia di transizioni fluide. Il backoffice è un CRUD classico dove Blade + Alpine.js è più semplice e performante. Inertia era stato installato inizialmente ma rimosso perché creava un livello intermedio inutile.

Q: Quali design pattern hai usato e perché?

5 pattern principali: (1) **Service Layer** — logica business separata da controller (NasalImageService, WordMapService, ImageUploadService). (2) **Observer** — side effects post-evento (CorpoCelesteObserver dispatcha job). (3) **Policy/Gate** — authorization centralizzata (bypass admin con `before()`). (4) **FormRequest** — validazione separata dal controller (13 FormRequest). (5) **Factory** — dati test strutturati. Non ho usato Repository Pattern: progetto troppo piccolo, avrebbe aggiunto astrazione inutile. Single Responsibility Principle come guida.

Auth & Authorization

Q: Come funziona l'autenticazione?

Laravel Breeze. Route protette da middleware `auth`. Login, register, profilo, reset password — tutto Blade. Admin demo: `admin@astralis.it` / `password`.

Q: Traccia il flusso end-to-end del login (form session)

6 step: (1) Form POST a `/login` con email + password + `_token` CSRF. (2) `Auth::attempt($credentials)` verifica hash password. (3) Se OK, session ID generato e salvato nel DB (`sessions` table). (4) Session cookie inviato al browser. (5) Ogni richiesta successiva manda il cookie `auth` middleware legge la sessione `$request->user()` restituisce l'utente. (6) "Remember me" cookie largo (30 giorni) invece di session cookie.

Q: Qual è la differenza tra autenticazione e autorizzazione?

Autenticazione = "chi sei?" Breeze, login, session, cookie. **Autorizzazione** = "cosa puoi fare?" Policy, Gate, `before()`. Astralis usa Breeze per auth, 5 Policy + 1 Gate per authZ.

Q: Come funziona l'autorizzazione?

Tre livelli: (1) middleware `auth` sulle route. (2) 5 Policy (una per entità) con metodo `before()` che bypassa tutto per admin. (3) Gate `admin` in `AuthServiceProvider` per controller senza policy.

Q: Policy vs Gate?

Policy: gruppo di permessi su un modello (es. `CategoriaPolicy` `viewAny`, `create`, `update`, `delete`). Gate: singola azione (es. Gate `admin`). Policy si usa con `$this->authorize('create', Categoria::class)`. Gate con `Gate::authorize('admin')`.

Q: Come funziona il pattern `before()` nelle Policy?

`before()` restituisce `?bool`. Se `true` bypassa tutto (admin). Se `null` controlla il metodo specifico (non-admin). Se `false` blocca tutto.

Database

Q: Quali relazioni hai implementato?

3 tipi: BelongsTo/HasMany (1-N): Categoria CorpoCeleste, CorpoCeleste Galleria, CorpoCeleste Curiosità. BelongsToMany (N-N): CorpoCeleste Missione con pivot corpo_celeste_missione (tipo_esplorazione, anno_arrivo).

Q: Come gestisci gli slug?

spatie/laravel-sluggable . Trait Sluggable nel modello + metodo getSlugOptions() . Genera slug dal nome italiano (nome). 3 modelli: Categoria, CorpoCeleste, Missione.

Q: Cos'è il problema N+1?

Quando in un loop accedi a una relazione senza eager loading. Esempio: 10 missioni × 1 corpo = 11 query. Soluzione: with(['categoria', 'galleria']) 2 query.

Q: Come fai il seeding?

database/seeder/ . Seeder con dati reali: 8 categorie, 18 corpi celesti, 10 missioni, 16 immagini galleria, 18 curiosità, 17 relazioni pivot. php artisan migrate:fresh --seed .

Q: Qual è la differenza tra soft delete e hard delete?

Hard delete (Astralis): delete() elimina la riga dal DB. cascadeOnDelete elimina anche le righe figlie. Irreversibile. **Soft delete**: SoftDeletes trait imposta deleted_at con timestamp. La riga resta nel DB ma non appare nelle query normali. withTrashed() la include, restore() la ripristina, forceDelete() la elimina davvero. Astralis usa hard delete perché il progetto non necessita di recovery.

Backend

Q: Cos'è un FormRequest?

Classe dedicata alla validazione, separata dal controller. 13 FormRequest nel progetto (Store/Update per ogni entità + SuggestNome, AggiornaOrdine, ProfileUpdate). Regole in rules() .

Q: Cos'è un Model e come lo crei?

Rappresenta una tabella del DB. Ogni Model ha \$fillable (campi modificabili), \$casts (tipi), relazioni come metodi. php artisan make:model CorpoCeleste -m crea modello + migration. In Astralis: 5 Model (Categoria, CorpoCeleste, Missione, Curiosità, GalleriaCorpo) in app/Models/ .

Q: Cos'è un Controller e dove lo trovi?

Classe che gestisce la logica HTTP: riceve la richiesta, chiama service/model, restituisce risposta. 7 metodi standard: index, create, store, show, edit, update, destroy. In Astralis: Admin controllers in app/Http/Controllers/Admin/ (8 file), API controllers in app/Http/Controllers/Api/ (6 file). php artisan make:controller .

Q: Che cos'è Blade?

Template engine di Laravel. {{ \$var }} stampa escaped. @if , @foreach , @extends , @section , @include . Componenti con <x-nome /> . In Astralis: layout admin in resources/views/admin/layouts/app.blade.php , guest SPA shell in resources/views/guest.blade.php .

Q: Quali middleware usi?

`auth` (login obbligatorio), `verified` (email verificata), `throttle:120,1` (rate limit), `throttle:30,1` su `suggestNome`. Definiti in `bootstrap/app.php`. Il middleware `HandleCors` gestisce le richieste cross-origin.

Q: Che differenza c'è tra route web e API?

Dual rendering. `web.php` restituisce **HTML** (Blade admin + catch-all React SPA). `api.php` restituisce **JSON** (API consumate dal React SPA). Il frontend chiama `/api/corpi-celesti` dal browser, il Vite proxy (`vite.config.js`) inoltra a `localhost:8000`, il controller API restituisce JSON. Routing separato: web per server-side rendering, api per client-side.

Q: Spiega `routes/web.php` — cosa contiene?

58 righe, 4 sezioni:

1. **Home** — `GET /` carica la vista `guest` (React SPA shell)
2. **Profilo** — 3 route protette da `auth`: `edit`, `update`, `destroy` del profilo utente
3. **Blocco admin** — raggruppato con `prefix('admin')` + middleware `auth`, `verified`, `throttle:120,1`:
 - `GET /admin` dashboard
 - 5 `Route::resource` per le CRUD (categorie, corpi-celesti, missioni, curiosità, galleria)
 - Route custom: `suggest-nome` (auto-traduzione), `gallery-add`, `set-image`, `ordine galleria`
 - Route NASA import: `import-all` + import singola
4. **Catch-all** — `GET /{any}` qualsiasi URL non matching torna a `guest.blade.php`. Va dopo tutte le route specifiche altrimenti cattura tutto
5. `require __DIR__.'/auth.php'` include le route Breeze

`Route::resource` genera 7 route CRUD in automatico. `parameters(['corpi-celesti' => 'corpoCeleste'])` rinomina il parametro URL per il Route Model Binding.

Q: Spiega `routes/api.php` — cosa contiene?

26 righe. 10 endpoint JSON pubblici (nessuna auth), tutti dentro `throttle:60,1`:

Endpoint	Cosa fa
<code>GET /api/corpi-celesti</code>	Lista (con filtri query)
<code>GET /api/corpi-celesti/{slug}</code>	Dettaglio (Route Model Binding su slug)
<code>GET /api/corpi-celesti/{slug}/simili</code>	Corpi simili
<code>GET /api/categorie</code>	Lista categorie
<code>GET /api/categorie/{slug}</code>	Dettaglio categoria
<code>GET /api/missioni</code>	Lista missioni
<code>GET /api/missioni/{slug}</code>	Dettaglio missione

GET /api/curiosita	Lista curiosità
GET /api/galleria	Lista galleria
GET /api/dashboard/stats	Statistiche dashboard admin

Route Model Binding con `{corpoCeleste:slug}` usa lo slug invece dell'ID. Il prefisso `/api` è automatico in Laravel. Le risposte passano da `ApiResponse` (trasforma il modello in JSON controllato). Throttle più aggressivo delle web (`60,1` vs `120,1`).

Q: Cos'è un Job e come lo usi?

Classe che esegue un task in background (queue). In Astralis: `ImportNasalmage` importa immagini da NASA API. Dispatchato da `CorpoCelesteObserver::created()` . `ShouldBeUnique` evita duplicati. `failed()` gestisce errori. `php artisan make:job` .

Q: Qual è la differenza tra Job e Queue?

Job = classe PHP che definisce il lavoro da fare (es. `ImportNasalmage`). **Queue** = meccanismo che contiene i job in attesa (Redis, database, o `sync` in sviluppo). Il Job viene dispatchato INTO la Queue. Un Queue Worker prende i job dalla Queue e li esegue. In sviluppo: `QUEUE_CONNECTION=sync` (esegue subito). In produzione: `php artisan queue:work` (async).

Q: Come gestisci la cache?

`Cache::remember('key', ttl, callback)` memorizza il risultato per N secondi. In Astralis: dashboard cache (600s), suggestNome cache (3600s). Invalidata nei controller con `Cache::flush()` o `Cache::forget()` . La Facade `Cache` accede al `CacheManager` dal container.

Q: Cos'è un Observer?

Pattern che ascolta eventi Eloquent (created, updated, deleted). `CorpoCelesteObserver::created()` dispatcha il job `ImportNasalmage` quando un corpo viene creato. Disabilitato in testing con `app()->environment('testing')` .

Q: Cos'è un Service Layer?

Classe che gestisce logica di business, separata dal controller. 3 service: `NasalmageService` (ricerca/import/dedup NASA), `WordMapService` (traduzione IT EN), `ImageUploadService` (upload + resize con Intervention Image v4). Riutilizzabili da controller, CLI, o observer.

Q: Come funziona l'upload?

`ImageUploadService` centralizza la logica. Intervention Image v4: `new ImageManager(new Driver())->decodePath($path)` . `scaleDown(1200, 1200)` preserva aspect ratio. Salvato su `storage/app/public/` con `Storage::disk('public')` . `php artisan storage:link` per il symlink.

Q: Traccia il flusso end-to-end dell'upload (form database)

6 step: (1) Form con `enctype="multipart/form-data"` (obbligatorio per file). (2) `$request->file('immagine')` restituisce un `UploadedFile` . (3) `ImageUploadService::upload($file)` validazione (mimes, max 5MB). (4) `scaleDown(1200, 1200)` ridimensiona preservando aspect ratio. (5) `Storage::disk('public')->put($path, $content)` salva il file. (6) Path restituito salvato nella colonna DB (es. `$corpoCeleste->update(['immagine' => $path])`).

Q: Traccia il flusso end-to-end dell'integrazione NASA

Chain completa: (1) Admin clicca "Importa da NASA". (2) `NasalImportController::importAll()` chiama `NasalImageService::searchAndImport()`. (3) `WordMapService::translate()` traduce nome IT EN (~70 termini astronomici). (4) NASA Image API chiamata con query tradotta. (5) Dedup per `nasa_id` (non duplica). (6) URL remoto salvato (non file locale). (7) `CorpoCelesteObserver::created()` si attiva `dispatcha ImportNasalimage job`. (8) Il job fetcha immagini aggiuntive dalla stessa API. `ShouldBeUnique` evita job duplicati.

Q: CORS come lo gestisci?

Laravel 13 gestisce automaticamente (middleware `HandleCors`). In dev, Vite proxy inoltra `/api` a `localhost:8000`. Nessun `config/cors.php` necessario.

Q: Cos'è un API Resource?

Trasforma un modello Eloquent in JSON controllato. `CorpoCelesteResource` espone nome (italiano) ma non campi interni come `immagine_utente`. `php artisan make:resource`.

Q: Come funziona il routing lato client con React?

`react-router-dom` definisce 5 route: `/`, `/corpi-celesti`, `/corpi-celesti/:slug`, `/confronta`, `/*` (404). Laravel cattura tutto con `Route::get('/{any}', fn() => view('guest'))->where('any', '.*')` e passa il controllo a React.

Flusso end-to-end del catch-all SPA React (6 step):

1. **Utente digita URL** — es. `localhost:5173/corpi-celesti/terra`
2. **Laravel route match** — `web.php` cerca una route che matcha. Se è `/admin/*` Blade admin. Se non matcha nulla cade nel catch-all `{any}`
3. **Catch-all restituisce HTML** — `return view('guest')` carica `guest.blade.php`, che contiene `<div id="guest-root">` + tag `<script>` di Vite (React bundle)
4. **React monta** — `main.jsx` esegue `ReactDOM.createRoot(document.getElementById('guest-root')).render(<App />)`. La SPA è viva nel browser
5. **react-router-dom legge l'URL** — `App.jsx` ha `<BrowserRouter>` con 5 `<Route>`. L'URL `/corpi-celesti/terra` matcha `/corpi-celesti/:slug` monta `<CorpoDettaglio />`
6. **Componente fetcha dati** — `CorpoDettaglio` chiama `fetchCorpoCeleste('terra')` `apiClient.js` invia `GET /api/corpi-celesti/terra` Vite proxy inoltra a `localhost:8000` API restituisce JSON React renderizza

Perché `{any}` DEVE stare in fondo? — Se fosse prima delle route admin, catturerebbe `/admin` prima che Laravel la processi e restituirebbe sempre la SPA.

Code splitting — `React.lazy()` + `<Suspense>` caricano ogni pagina solo quando l'utente ci naviga. Se navigo a `/corpi-celesti`, il bundle di `Comparatore` non viene scaricato.

ScrollToTop — Ogni cambio route resetta lo scroll in cima (altrimenti resti dove eri nella pagina precedente).

404 — Route `path="*" in React` mostra `<NotFound />` . Laravel non arriva mai a generare un 404 server-side perché il `catch-all` cattura tutto.

Frontend

Q: Come comunica React con Laravel?

`apiClient.js` (axios) invia richieste a `/api/*` . Vite proxy inoltra a `localhost:8000` . API REST restituiscono JSON. Nessun Inertia (rimosso).

Q: Cos'è axios e perché lo usi?

Axios è una libreria HTTP per JavaScript. In Astralis la usiamo al posto di `fetch()` nativo perché: (1) gestisce automaticamente il parsing JSON, (2) ha interceptor per errori centralizzati, (3) supporta timeout nativo, (4) il nostro `apiClient.js` ha retry logic + backoff esponenziale. Installata con `npm install axios` . Tutto passa da `apiClient.js` — nessun componente usa `fetch()` direttamente.

Q: Dove si configura axios?

`resources/js/guest/apiClient.js` . Configurazione: `baseUrl: '/api'` , `timeout: 30000` , header `Accept: application/json` . L'interceptor gestisce retry (2 tentativi, backoff esponenziale) e logging errori in DEV. I 5 export funzioni (`fetchCorpiCelesti` , `fetchCorpoCeleste` , ecc.) wrappano le chiamate `apiClient` con `signal` per `AbortController`.

Flusso errore end-to-end in Astralis — cosa succede quando un'API fallisce:

Caso 1: 404 Not Found (slug inesistente)

- Axios riceve risposta `404` interceptor la passa al componente
- `useFetch` dispatcha `ERROR` il componente mostra fallback UI
- In Astralis: `CorpoDettaglio` mostra "Corpo celeste non trovato" con icona Rocket + link "Torna alla lista"

Caso 2: 403 Forbidden (non-admin tenta CRUD)

- Policy `before()` restituisce `false` Laravel restituisce 403
- Axios riceve 403 interceptor non fa retry (4xx non retryabile)
- `useFetch` dispatcha `ERROR` redirect a login o messaggio "Accesso negato"

Caso 3: 500 Server Error (bug, cache rotta, DB down)

- Axios riceve 500 interceptor **riprova automaticamente** (backoff esponenziale, max 2x)
- Se dopo 2 retry fallisce ancora `useFetch` dispatcha `ERROR`
- Il componente mostra messaggio generico "Qualcosa è andato storto"

Caso 4: Timeout di rete (server non risponde)

- Axios timeout (30s) errore senza `error.response`
- Interceptor lo tratta come retryabile (stessa logica del 500)

***Error Boundary** cattura solo crash JavaScript (errore nel rendering), NON errori HTTP. Se un componente crasha, mostra fallback UI; se l'API fallisce, lo gestisce `useFetch`.*

Q: Cos'è un Error Boundary?

Class component React che cattura errori nei figli. Mostra fallback UI se un componente crasha. In `App.jsx` avvolge `<Routes>`.

Q: Cos'è React.memo()?

Ottimizzazione: il componente non re-renderizza se le props non cambiano. Usato su `LightboxGalleria` e `Thumbnail`.

Q: Come gestisci il fetch dei dati?

Custom hook `useFetch` con `useReducer` + `AbortController`. Gestisce loading, error, e cleanup. `apiClient.js` ha retry logic (2 tentativi).

Q: Come avete implementato il Sistema Solare animato?

Soluzione finale con `requestAnimationFrame` + direct DOM manipulation. Un angolo cresce infinitamente per ogni pianeta, convertito in coordinate x/y con funzioni seno/coseno. Ogni pianeta ha velocità e raggio differenziati. Hover rallenta i pianeti al 33%. Zero re-render React durante animazione.

Q: Perché avete salvato URL remoti NASA invece di file locali?

Le immagini NASA in risoluzione originale possono essere enormi. Salvando l'URL remoto `~medium.jpg`: (1) risparmio storage, (2) no download enormi, (3) browser carica da NASA CDN. Il comando `astralis:gallery --fix` sostituisce URL non raggiungibili con placeholder.

Q: Che cos'è il WordMapService e a cosa serve?

Servizio di traduzione italiano - inglese parola per parola, con ~70 termini astronomici. Quando un admin inserisce un nome italiano, il WordMapService traduce la query prima di cercare su NASA API. Cache con TTL di 1 ora.

Testing

Q: Come testi le API?

PHPUnit + `Http::fake()`. Ogni test crea dati con factory, chiama l'endpoint, verifica status e JSON response. `Http::fake()` blocca chiamate HTTP reali.

Q: Perché Http::fake() è essenziale?

L'Observer dispatcha il job NASA ogni volta che un `CorpoCeleste` viene creato. Senza `Http::fake()`, i test farebbero chiamate reali.

Q: Come testi React?

Vitest + Testing Library. 110 test: componenti, integrazione API, error handling. `jsdom` environment.

Q: Quali sono i pattern obbligatorio per i test?

`Http::fake()` in `setUp()`, `RefreshDatabase` per DB pulito, `factory()->create()` invece di

inserimenti manuali, Observer disabilitato in testing.

Deployment

Q: Come funziona il deployment in produzione?

Checklist: (1) `.env` `APP_ENV=production` , `APP_DEBUG=false` , `APP_URL` . (2) `php artisan config:cache` + `php artisan route:cache` (cache compilata, più veloce). (3) `php artisan migrate` (applica migrazioni). (4) `php artisan storage:link` (symlink). (5) `npm run build` (Vite produce bundle minificato in `public/build/`). (6) Queue workers: `php artisan queue:work` (processa job async). (7) Vite proxy non serve (stesso dominio). (8) Cron per scheduler: `* * * * * php artisan schedule:run` .

7. Definizioni PHP

Classe e Oggetto

```
class CorpoCeleste extends Model { ... } // Classe = blueprint
$corpo = CorpoCeleste::find(1);      // Oggetto = istanza
```

La classe definisce cosa un oggetto può fare. L'oggetto è l'istanza concreta.

Namespace e Use

```
namespace App\Models;           // Organizza il codice in cartelle
use Illuminate\Database\Eloquent\Model; // Importa una classe
```

Namespace = indirizzo della classe. Use = importazione per usarla senza scrivere il percorso completo.

Trait

```
trait HasFactory {
    public static function factory() { ... }
}

class CorpoCeleste extends Model {
    use HasFactory; // Aggiunge metodi del trait al modello
}
```

Trait = codice riutilizzabile che si "include" in una classe. Simile all'ereditarietà ma multipla.

Type Hints e Return Types

```
public function categoria(): BelongsTo { ... } // Return type
public function before(User $user): ?bool { ... } // Param + return
public function getNomeAttribute(): string { ... } // Accessor con return type
```

Type hints = indica il tipo di parametro. Return type = indica il tipo di valore restituito. `?bool` = null o bool.

Union Types (PHP 8.0)

```
function getNome(string|bool $fallback): string { ... } // Accetta string O bool
function findById(int|string $id): ?Model { ... } // int O string
```

Un parametro può essere di più tipi con `|`. Più preciso di `mixed`, più flessibile di un singolo tipo. In Laravel: usato nei Service Container, nelle API, nelle validation rules.

Array Associativo

```
$stats = [
    'corpi' => CorpoCeleste::count(), // Chiave => Valore
    'categorie' => Categoria::count(),
];
echo $stats['corpi']; // 18
```

Array con chiavi stringa invece di indici numerici.

Public, Private, Protected

```
public $name; // Accessibile ovunque
protected $email; // Accessibile nella classe e nelle figlie
private $secret; // Accessibile solo nella classe
```

Static Method

```
CorpoCeleste::count(); // Metodo statico: si chiama sulla classe, non sull'oggetto
```

Include e Require


```
include 'config.php'; // Includi il file. Se fallisce warning, continua
require 'config.php'; // Includi il file. Se fallisce fatal error, stop
include_once 'config.php'; // Come include, ma solo 1 volta (evita doppie dichiarazioni)
require_once 'config.php'; // Come require, ma solo 1 volta
```

Differenza include vs require: `include` è permissivo (continua anche se il file manca), `require` è obbligatorio (script si ferma). **Differenza_once:** garantisce che il file venga caricato una sola volta. In Laravel non li usi direttamente — Composer gestisce l'autoloading con `require_once` nel file `vendor/autoload.php`.

Closure e Arrow Function

```
// Closure classica
$callback = function ($value) {
    return $value * 2;
};

// Arrow function (PHP 7.4+) — più concisa, cattura variabili dallo scope automaticamente
$moltiplica = fn($value) => $value * 2;

// In Laravel: route, callback, higher-order
Route::get('/test', fn() => view('test'));
$corpi->filter(fn($c) => $c->in_evidenza);
```

Closure = funzione anonima. Arrow function (`fn()`) è la versione concisa: una sola riga, return implicito, cattura automatica dello scope.

Enum

```
// PHP 8.1+ — Enum a stringhe (backed enum)
enum TipoCorpoCeleste: string {
    case Pianeta = 'pianeta';
    case Stella = 'stella';
    case Galassia = 'galassia';
    case Nebulosa = 'nebulosa';
}

// Uso
$tipo = TipoCorpoCeleste::Pianeta;
$tipo->value; // 'pianeta'
$tipo->name; // 'Pianeta'
```

```
// Match con enum
$descrizione = match($tipo) {
    TipoCorpoCeleste::Pianeta => 'Corpo celeste che orbita attorno a una stella',
    TipoCorpoCeleste::Stella  => 'Corpo celeste che emette luce',
    default                   => 'Altro',
};
```

Enum = enumerazione di valori costanti. In PHP 8.1+ sono classi con vincolo sui valori (string o int). Backed enum hanno `.value` e `.name`. Più sicuro di stringhe libere — il tipochecked a compile time. In Astralis: i tipi di corpo celeste sono stringhe nel DB (`$fillable`), ma un enum è lo scenario ideale per status, stati, tipi fissi.

Null Safe Operator (`?->`)

```
$nome = $corpo->categoria?->nome;    // Se categoria è null   null (non errore)
$icona = $corpo->categoria?->icona ?? '—'; // Con fallback
```

Operatore di navigazione sicura. Se l'oggetto a sinistra è `null`, restituisce `null` invece di lanciare errore. Introdotto in PHP 8.0. Utile dove la relazione potrebbe essere null.

::class

```
Categoria::class    // Restituisce la stringa completa: 'App\Models\Categoria'
Route::resource('categorie', CategoriaController::class);
```

`::class` è una costante magica che restituisce il nome fully-qualified della classe. Utile per route, policy, e dove Laravel richiede una stringa che identifichi univocamente una classe.

Ereditarietà

```
class CorpoCeleste extends Model { ... }
class CorpoCelesteController extends Controller { ... }
```

Una classe figlia eredita tutti i metodi e proprietà pubbliche/protette del padre. `extends` = eredita. `override` = sovrascrive un metodo del padre. PHP supporta ereditarietà singola (una sola classe padre) + trait multipli.

Facade

```
Cache::remember('corpi', 3600, fn() => CorpoCeleste::all());
```

```
Gate::authorize('admin');
```

Una Facade è un “proiettore statico” che accede a un servizio dal container di Laravel. Dietro `Cache::` c'è un'istanza reale di `CacheManager`. Facade = sintassi compatta + testabilità (si possono mockare nei test con `Cache::shouldReceive()`).

String Functions

```
strlen('hello')      // 5 — lunghezza stringa
strtoupper('hello')  // 'HELLO' — maiuscolo
strtolower('HELLO')  // 'hello' — minuscolo
str_replace('a', 'x', 'banana') // 'bxnxbn' — sostituzione
substr('hello', 0, 3) // 'hel' — sottostringa
strpos('hello world', 'world') // 6 — posizione (false se non trovato)
trim(' hello ')      // 'hello' — rimuove spazi
str_contains('hello world', 'world') // true (PHP 8+)
str_starts_with('hello', 'he')      // true (PHP 8+)
```

Funzioni base per manipolare stringhe. Non sono metodi di una classe — sono funzioni globali. In Laravel: usate ovunque (validation, slug, formattazione).

Array Functions

```
count($array)      // Numero elementi
array_push($arr, 'x') // Aggiunge in fondo
array_pop($arr)     // Rimuove dall'ultimo
array_merge($a, $b) // Unisce due array
array_key_exists('key', $arr) // Verifica chiave
in_array('value', $arr) // Verifica valore
array_map(fn($x) => $x * 2, $arr) // Applica funzione a ogni elemento
array_filter($arr, fn($x) => $x > 5) // Filtra per condizione
array_column($arr, 'nome') // Estrae una colonna
compact('var1', 'var2') // Crea array da variabili
```

Funzioni globali per manipolare array. In Laravel: usate in controller, seeder, test. Le Collection Eloquent estendono queste funzionalità con metodi fluenti.

Null Coalescing (??)

```
$nome = $input['nome'] ?? 'Anonimo'; // Se $input['nome'] non esiste o è null 'Anonimo'
$valore = $x ?? $y ?? $z ?? 'default'; // Primo valore non-null
```

```
$risultato = $request->input('campo', 'default'); // Laravel usa ?? internamente
```

Operatore introdotto in PHP 7. Simile al ternario `?:` ma più sicuro: restituisce il valore a destra solo se il sinistro è `null` o non esiste. Differenza con `isset()` : `??` controlla solo `null` , non `false` o `0` .

Type Casting

```
$valore = (int) '123';    // 123 — intero
$valore = (string) 123;   // '123' — stringa
$valore = (float) '3.14'; // 3.14 — float
$valore = (bool) 0;       // false — booleano
$valore = (array) $oggetto; // Array dall'oggetto

// In Model Eloquent: $casts
protected $casts = [
    'in_evidenza' => 'boolean',
    'temperatura' => 'integer',
    'data_lancio' => 'date',
];
```

Conversione esplicita di tipo con parentesi. In PHP: `(int)` , `(string)` , `(bool)` , `(float)` , `(array)` , `(object)` . In Laravel: `$casts` del Model fa la conversione automatica dal database.

Date & Carbon

```
use Carbon\Carbon;

$oggi = Carbon::now();           // Data e ora attuale
$ieri = Carbon::yesterday();
$domani = Carbon::tomorrow();

$now->addDays(5);                 // +5 giorni
$now->subHours(3);                 // -3 ore
$now->diffInDays($altra_data);    // Differenza in giorni
$now->format('d/m/Y H:i');        // Formattazione

Carbon::parse('2024-01-15')->isPast(); // true
Carbon::parse('2025-12-25')->isFuture(); // true
```

Carbon è il package di manipolazione date di Laravel. Estende `DateTime` di PHP. In Astralis: usato nelle date delle missioni (`data_lancio`), nei timestamp delle migration. Le migration usano

`now()` che restituisce un'istanza Carbon.

== vs === (Loose vs Strict Comparison)

```
// Loose (==) — converte i tipi prima di confrontare
1 == "1"    // true  — stringa "1" diventa intero 1
0 == ""     // true  — entrambi falsy
null == false // true

// Strict (===) — confronta anche il tipo
1 === "1"   // false — intero vs stringa
0 === ""    // false — intero vs stringa
null === false // false — null vs bool
```

Regola pratica: usare **sempre** `===` (strict) per evitare sorprese. In PHP 8.0+ il comportamento di `==` è cambiato (es. `0 == "foo"` ora è `false`), ma `===` resta prevedibile.

isset() vs empty()

```
$nome = null;
$eta = 0;
$lista = [];

isset($nome); // false — la variabile è null
isset($eta);  // true  — esiste e non è null
empty($nome);  // true  — è "vuoto" (null, false, 0, "", [])
empty($eta);   // true  — 0 è considerato "vuoto" da empty()
empty($lista); // true  — array vuoto è "vuoto"
```

Differenza chiave: `isset()` controlla solo se è `null`. `empty()` controlla se è “falsy” (null, false, 0, "", []). In Laravel: `$request->filled('nome')` è l'opposto di `empty()` per i request input.

array_push() vs array_merge()

```
// array_push — aggiunge elementi in fondo all'array
$frutti = ['mela', 'pera'];
array_push($frutti, 'banana', 'arancia');
// ['mela', 'pera', 'banana', 'arancia']

// array_merge — unisce due array (indici numerici vengono rinumerati)
$a = ['mela', 'pera'];
$b = ['banana', 'arancia'];
```

```
$uniti = array_merge($a, $b);
// ['mela', 'pera', 'banana', 'arancia']

// Con array associativi, array_merge sovrascrive le chiavi duplicate
$config = ['timeout' => 30, 'retries' => 2];
$override = ['timeout' => 60];
$finale = array_merge($config, $override);
// ['timeout' => 60, 'retries' => 2]
```

Quando usare cosa: `array_push` per aggiungere elementi uno alla volta. `array_merge` per unire array completi. In Laravel: `array_merge` è usato nei config per override dei default.

str_contains() vs strpos()

```
// str_contains (PHP 8+) — restituisce bool, leggibile
str_contains('Via Lattea', 'Lattea'); // true
str_contains('Via Lattea', 'Marte'); // false

// strpos (pre-PHP 8) — restituisce la posizione (0 = trovato all'inizio), false se non trovato
strpos('Via Lattea', 'Lattea') !== false; // true
strpos('Via Lattea', 'Marte') !== false; // false
```

Regola: in PHP 8+ usare sempre `str_contains()`. Più leggibile e meno errori (evita il bug classico `strpos() == false` che confonde posizione 0 con “non trovato”). Analogamente: `str_starts_with()`, `str_ends_with()`.

Funzioni di Debug: var_dump, print_r, dd

```
// var_dump — mostra tipo + valore di tutto
var_dump($corpoCeleste);
// object(CorpoCeleste)#1 (23) { ["id"]=> int(3) ["nome"]=> string(5) "Terra" ... }

// print_r — mostra solo il valore, più leggibile per array
print_r($stats);
// Array ( [corpi] => 18 [categorie] => 8 )

// dd() — Laravel: dump + die (ferma l'esecuzione)
dd($corpoCeleste); // Stampa tutto e STOPPA il programma
// dump() — Laravel: dump senza morire (continua l'esecuzione)
```

In Laravel: `dd()` è il più usato per debugging rapido. `dump()` quando vuoi vedere il valore senza interrompere. In React: `console.log()` equivale a `dump()`.

Variadic Arguments (...\$args)

```
// Funzione che accetta N argomenti
function somma(...$numeri): int {
    return array_sum($numeri);
}

somma(1, 2, 3);    // 6
somma(10, 20);    // 30

// Spread operator (l'opposto) — espande un array in argomenti
$corpi = ['Terra', 'Marte', 'Giove'];
$frutta = ['mela', 'pera', 'banana'];
$uniti = [...$corpi, ...$frutta];
// ['Terra', 'Marte', 'Giove', 'mela', 'pera', 'banana']
```

In PHP 7.4+ lo spread operator funziona anche sugli array. In Laravel: usato nei test per passare argomenti multipli e nei config per mergare array.

Named Arguments (PHP 8.0)

```
// Posizionale (classico) — difficile leggere con molti parametri
htmlspecialchars($string, ENT_QUOTES, 'UTF-8', false);

// Named (PHP 8.0) — chiaro e skipa parametri opzionali
htmlspecialchars($string, double_encode: false, encoding: 'UTF-8');
```

PHP 8.0 permette di passare argomenti per nome: `paramName: value` . Vantaggi: leggibilità, skipa parametri con default, non dipende dall'ordine. In Laravel: usato nei config, nelle migration, nelle API. Esempio reale: `Route::get('/test', controller: 'App\Http\Controllers\TestController')` .

Match Expression (PHP 8.0)

```
// switch classico — verbose, comparison non strict
switch ($stato) {
    case 'completata': $icona = 'check'; break;
    case 'in_corso':  $icona = 'clock'; break;
    default:         $icona = 'help'; break;
}

// match (PHP 8.0) — conciso, strict comparison, restituisce valore
$icona = match($stato) {
```

```
'completata' => 'check',  
'in_corso' => 'clock',  
default => 'help',  
};
```

`match()` è il sostituto moderno di `switch` : confronto **strict** (`===`), restituisce un valore, non serve `break` . Più sicuro e conciso. In Laravel: usato al posto di `switch` per conversioni dati, status mapping, valori condizionali.

Constructor Property Promotion (PHP 8.0)

```
// Prima di PHP 8.0 — boilerplate ripetitivo  
class MissioneController {  
    protected $service;  
    protected $cache;  
    public function __construct(NasalImageService $service, CacheManager $cache) {  
        $this->service = $service;  
        $this->cache = $cache;  
    }  
}  
  
// PHP 8.0 — unica riga definisce proprietà + assegna  
class MissioneController {  
    public function __construct(  
        protected NasalImageService $service,  
        protected CacheManager $cache,  
    ) {}  
}
```

PHP 8.0 permette di dichiarare e assegnare proprietà direttamente nei parametri del costruttore. Riduce boilerplate. Modificatori (`public` , `protected` , `private`) determinano la visibilità. In Laravel 11+: usato nei controller, service, request.

Abstract Classes

```
abstract class BaseController {  
    // Metodo concreto — ereditato così com'è  
    protected function formatResponse($data): array {  
        return ['success' => true, 'data' => $data];  
    }  
  
    // Metodo astratto — ogni figlio DEVE implementarlo
```



```
abstract protected function getValidatorRules(): array;
}
```

Classe che **non può essere istanziata** direttamente — serve solo da base per le classi figlie. Contiene metodi concreti (ereditati) e metodi astratti (obbligatori nei figli). `extends` per ereditare. In Laravel: `Controller` base è abstract, i controller concreti la estendono. Utile per logica condivisa (response format, validation pattern).

Interfaces

```
// Definizione — contratto che la classe DEVE implementare
interface HasThumbnail {
    public function getThumbnailUrl(): string;
    public function regenerateThumbnail(): bool;
}

// Implementazione — obbligo di tutti i metodi dell'interfaccia
class CorpoCeleste extends Model implements HasThumbnail {
    public function getThumbnailUrl(): string {
        return $this->immagine_url;
    }
    public function regenerateThumbnail(): bool {
        return ImageUploadService::regenerate($this);
    }
}
```

Interface = contratto: definisce metodi che una classe **deve implementare**. `implements` per applicare. Differenza da abstract class: una classe può implementare **più interfacce** ma estendere **una sola classe**. In Laravel: `ShouldQueue` (Job), `HasFactory` (Model), `Jsonable`, `Arrayable`. Garantisce compatibilità tra classi diverse.

Readonly Properties (PHP 8.1)

```
class NasalImageService {
    // Assegnato una volta nel costruttore, mai modificabile dopo
    private readonly WordMapService $wordMap;

    public function __construct(WordMapService $wordMap) {
        $this->wordMap = $wordMap;
    }

    public function translate(string $nome): string {
```

```
return $this->wordMap->translate($nome); // OK
// $this->wordMap = new WordMap(); // ERRORE: readonly
}
}
```

`readonly` = la proprietà può essere assegnata **solo nel costruttore**. Qualsiasi tentativo di modifica dopo l'istanziatura lancia un errore. Garantisce immutabilità e previene bug da stato mutabile. In Astralis: `NasalImageService.php` usa `private readonly WordMapService $wordMap` .

8. Definizioni Laravel

Model

Classe che rappresenta una tabella del database. Ogni riga = un'istanza. `$fillable` = campi modificabili in massa. `$casts` = conversione tipi. Relazioni definite come metodi (`belongsTo`, `hasMany`, `belongsToMany`). `php artisan make:model CorpoCeleste -m` crea modello + migration. In Astralis: 5 Model in `app/Models/` .

Accessors & Mutators

Accessor = metodo che formatta un campo quando viene letto. **Mutator** = metodo che modifica un campo quando viene scritto.

```
class CorpoCeleste extends Model {
    // Accessor — formatta il valore in lettura
    public function getImmagineUrlAttribute(): string
    {
        if (!$this->immagine) return '/images/default.png';
        if (str_starts_with($this->immagine, 'http')) return $this->immagine;
        return Storage::disk('public')->url($this->immagine);
    }

    // Mutator — modifica il valore in scrittura (Laravel 11+ syntax)
    public function setNomeAttribute(string $value): void
    {
        $this->attributes['nome'] = trim($value);
    }
}
```

```
// Uso: l'accessor viene chiamato automaticamente
$corpo->immagine_url; // Chiama getImmagineUrlAttribute()
```

Accessor/Mutator nascondono la logica di trasformazione dentro il Model. In Astralis: `getImmagineUrlAttribute()` gestisce automaticamente URL remoti, locali, e fallback.

Eloquent ORM

L'ORM di Laravel. Ogni tabella ha un Model. Relazioni definite come metodi. Query builder con catene di metodi.

Migration

File PHP che definisce/modify tabelle. `php artisan make:migration` . `up()` applica, `down()` annulla. Versionata con timestamp.

SQL Base — Concetti Fondamentali

Primary Key vs Foreign Key

Concetto	Cosa fa	Esempio in Astralis
Primary Key	Identifica ogni riga in modo univoco	<code>id</code> in ogni tabella (<code>bigIncrements</code>)
Foreign Key	Collega una tabella a un'altra, vincola l'integrità	<code>categoria_id</code> in <code>corpi_celesti</code> punta a <code>categorie.id</code>

```
// Primary Key — generata automaticamente da Laravel
$table->id(); // bigIncrements, auto-incrementale

// Foreign Key — garantisce che il valore esista nell'altra tabella
$table->foreignId('categoria_id')->constrained('categorie')->restrictOnDelete();
```

Senza FK: puoi inserire `categoria_id = 999` anche se la categoria 999 non esiste dati orfani.

Con FK: il database blocca l'inserimento integrità referenziale.

INNER JOIN vs LEFT JOIN

```
// INNER JOIN — restituisce SOLO le righe che hanno corrispondenza in entrambe le tabelle
CorpoCeleste::join('categorie', 'corpi_celesti.categoria_id', '=', 'categorie.id')
->select('corpi_celesti.nome', 'categorie.nome as categoria')
->get();

// Solo corpi che HANNO una categoria associata
```

```
// LEFT JOIN — restituisce TUTTE le righe della tabella sinistra, anche senza corrispondenza
Categoria::leftJoin('corpi_celesti', 'categorie.id', '=', 'corpi_celesti.categoria_id')
->select('categorie.nome', DB::raw('COUNT(corpi_celesti.id) as totale'))
->groupBy('categorie.nome')
->get();
// Tutte le categorie, anche quelle senza corpi (totale = 0)
```

Regola: usa `LEFT JOIN` quando vuoi TUTTE le righe della tabella principale. Usa `INNER JOIN` quando vuoi solo i record collegati. In Laravel: `withCount('corpiCelesti')` è l'equivalente elegante del `LEFT JOIN` con `COUNT`.

WHERE vs HAVING

```
// WHERE — filtra PRIMA del raggruppamento (su singole righe)
CorpoCeleste::where('in_evidenza', true)->get();
// Filtra i corpi celesti prima di qualsiasi raggruppamento

// HAVING — filtra DOPO il raggruppamento (su gruppi aggregati)
Categoria::select('categorie.nome', DB::raw('COUNT(corpi_celesti.id) as totale'))
->leftJoin('corpi_celesti', 'categorie.id', '=', 'corpi_celesti.categoria_id')
->groupBy('categorie.nome')
->having('totale', '>', 3)
->get();
// Solo categorie con più di 3 corpi celesti
```

Regola: `WHERE` = filtro sulle singole righe. `HAVING` = filtro sul risultato aggregato (`COUNT`, `SUM`, `AVG`). `WHERE` viene eseguito prima di `GROUP BY`, `HAVING` dopo.

GROUP BY — Aggregazione Dati

```
// Raggruppa per categoria e conta
$stats = CorpoCeleste::select('categoria_id', DB::raw('COUNT(*) as totale'))
->groupBy('categoria_id')
->get();
// [{categoria_id: 1, totale: 8}, {categoria_id: 2, totale: 3}, ...]

// GROUP BY + ORDER BY — categorie per numero di corpi (decrescente)
$stats = CorpoCeleste::select('categoria_id', DB::raw('COUNT(*) as totale'))
->groupBy('categoria_id')
->orderByDesc('totale')
->get();
```

In Astralis: il `DashboardController` usa `GROUP BY` per i 3 grafici: corpi per categoria (donut), corpi per tipo (barre), missioni per stato (barre orizzontali). SQL raw con `DB::raw()` per le

aggregazioni.

Seeder

File che popola il database. `DatabaseSeeder.php` chiama i seeder specifici. `php artisan db:seed` .

Factory

Classe che genera dati fake per test. `CorpoCeleste::factory()->create()` . Definita in `database/factories/` .

```
php artisan make:factory CorpoCelesteFactory
```

Observer

Classe che ascolta eventi Eloquent: `created` , `updated` , `deleted` , `saving` , ecc. Registrato in `AppServiceProvider::boot()` (Laravel 11+). Pattern: logica business separata dal controller.

```
php artisan make:observer CorpoCelesteObserver --model=CorpoCeleste
```

Events & Listeners

Sistema pub/sub di Laravel. Un **Event** è un “qualcosa è successo” (es. `UserRegistered`). Un **Listener** è “cosa fare quando succede” (es. `SendWelcomeEmail`). Sono diversi dagli Observer: gli Eventi sono classi PHP dedicate, gli Observer ascoltano eventi Eloquent.

```
// Definire un evento
class MissionCompleted { public function __construct(public Missione $missione) {} }

// Definire un listener
class NotifyAdmins { public function handle(MissionCompleted $event): void { ... } }

// Dispatch
event(new MissionCompleted($missione));

// Registrazione in EventServiceProvider (Laravel 10) o AppServiceProvider::boot()
```

In Astralis: gli Observer bastano (eventi Eloquent), ma per eventi custom (es. “missione completata”) si userebbe Events & Listeners. Il Observer è registrato in `AppServiceProvider::boot()` .

Service Layer

Classe che gestisce logica di business complessa. Separata dal controller (HTTP) e dal modello (DB). Riutilizzabile da controller, CLI, observer. Esempio: `NasalImageService` , `WordMapService` , `ImageUploadService` .

Policy

Gruppo di permessi per un modello. Metodi: `viewAny` , `view` , `create` , `update` , `delete` . `before()` per logica generale (es. admin bypassa tutto).

```
php artisan make:policy CorpoCelestePolicy --model=CorpoCeleste
```

Gate

Singola azione di autorizzazione. Definito in `AuthServiceProvider` . `Gate::define('admin', fn(User $user) => $user->is_admin)` . Usato con `Gate::authorize('admin')` .

FormRequest

Classe dedicata alla validazione. `rules()` restituisce array di regole. `messages()` per messaggi custom. Il controller la inietta come parametro.

```
php artisan make:request StoreCorpoCelesteRequest
```

Validation Rules

Regole di validazione per i campi. Si usano nei FormRequest (`rules()`) o direttamente nel controller (`$request->validate([...])`). Combinabili con `|` .

```
'nome' => 'required|string|max:255',  
'colore' => 'required|regex:/^[0-9A-Fa-f]{6}$/',  
'immagine_file' => 'nullable|image|max:2048',  
'email' => 'required|email|unique:users,email',  
'categoria_id' => 'required|exists:categorie,id',
```

Regole principali: `required` , `string` , `integer` , `max` , `min` , `email` , `unique` , `exists` , `nullable` , `image` , `regex` , `in` , `boolean` .

Middleware

Livello intermedio che filtra richieste. `auth` = verifica login. `verified` = email verificata. `throttle:120,1` = max 120 richieste/minuto. Definito in `bootstrap/app.php`.

Controller

Classe che gestisce la logica HTTP. Riceve la richiesta, interagisce con model/service, restituisce risposta (HTML o JSON). 7 metodi standard: index (lista), create (form), store (salva), show (dettaglio), edit (form modifica), update (aggiorna), destroy (elimina). `php artisan make:controller CorpoCelesteController -r`.

Resource Controller

Controller con 7 metodi standard. `Route::resource('categorie', CategoriaController::class)` genera tutte le route CRUD.

Artisan Commands

CLI di Laravel. `php artisan` + comando. Genera boilerplate, esegue task, gestisce il progetto. Comandi principali:

```
php artisan make:model CorpoCeleste -m # Model + Migration
php artisan make:controller -r # Resource Controller
php artisan make:request StoreXRequest # FormRequest
php artisan make:policy XPolicy --model # Policy
php artisan make:observer XObserver # Observer
php artisan make:job XJob # Job
php artisan make:factory XFactory # Factory
php artisan migrate # Esegue migration
php artisan migrate:fresh --seed # Reset DB + seed
php artisan serve # Avvia server locale
php artisan test # Esegue test PHPUnit
```

Comandi Artisan Custom

Puoi creare comandi custom con `php artisan make:command`. Ogni comando ha una `$signature` (nome + parametri) e un metodo `handle()`.

```
// app/Console/Commands/FetchNasaCommand.php
class FetchNasaCommand extends Command
{
    protected $signature = 'astralis:fetch-nasa {corpo?}'; // Parametro opzionale
    protected $description = 'Importa immagini da NASA API';
```

```

public function handle(): int
{
    $this->info('Importazione in corso...');
    // Logica del comando
    $this->info('Fatto!');
    return self::SUCCESS;
}
}

// Uso: php artisan astralis:fetch-nasa "Terra"

```

In Astralis: 3 comandi custom — `FetchNasaCommand` , `CleanupGalleryDuplicates` , `CheckFileHeaders` . Eseguibili con `php artisan astralis:nome-comando` .

Storage

Filesystem astratto. `Storage::disk('public')->put($path, $content)` . `storage/app/public/` per file accessibili web. `php artisan storage:link` per symlink.

Cache

Memorizzazione risultati costosi. `Cache::remember('key', 3600, fn() => $data)` . TTL in secondi. Redis, Memcached, o file.

Collection

Array potenziato di Laravel. `collect($array)` crea una collection. Metodi: `filter()` , `map()` , `pluck()` , `sum()` , `avg()` , `first()` , `sortBy()` , `groupBy()` , `toArray()` . Catena di metodi fluente.

```

$nomi = collect($corpi)
    ->filter(fn($c) => $c->in_evidenza)
    ->pluck('nome')
    ->toArray();

$somma = collect($prodotti)->sum('prezzo');
$media = collect($prodotti)->avg('prezzo');

```

L'API Rest di Laravel restituisce sempre Collection. Eloquent `->get()` restituisce `Collection` .

Blade

Template engine di Laravel. `{{ $variabile }}` stampa escaped. `@if` , `@foreach` , `@extends` , `@section` , `@include` . Componenti con `<x-nome />` .

@include (Partial)

Inserisce il contenuto di un altro file Blade. Passa variabili con il secondo parametro array. Eredita tutte le variabili dal padre. Utile per pezzi di layout riutilizzati: header, sidebar, form, badge, azioni.

```
@include('admin.partials.back-link', ['route' => 'admin.corpi-celesti.index'])
@include('admin.partials.stat-card', ['label' => 'Tipo', 'value' => $corpo->tipo ?? '—'])
```

In Astralis: 82 `@include` nell'admin — back-link, search, stat-card, category-badge, index-actions, empty-table-row, dashboard-stat, `_form`.

Componente Blade (`<x-...>`)

Componente riutilizzabile con props definite. Differenza da `@include` : ha scope isolato, props dichiarate con `@props` , riuso come tag HTML.

```
<x-guest-layout>      <!-- Layout auth Breeze -->
<x-app-layout>        <!-- Layout admin con sidebar -->
```

In Astralis: usato solo nelle pagine auth Breeze e profilo. L'admin usa `@include` per i partiali.

Differenza: Partial vs Componente

	Partial (<code>@include</code>)	Componente (<code><x-...></code>)
Sintassi	<code>@include('path')</code>	<code><x-nome /></code>
Variabili	Esplicite nell'array	<code>@props</code> dichiarati
Scope	Eredita tutto dal padre	Isolato
Riuso	Layout pezzi (form, badge, azioni)	UI atoms (layout, input, button)

Route Model Binding

Laravel risolve automaticamente il modello dalla route. `{corpoCeleste:slug}` usa lo slug invece dell'ID. `{corpoCeleste}` usa l'ID di default. Il controller riceve l'istanza del modello, non l'ID.

```
// Route: Route::get('/corpi-celesti/{corpoCeleste:slug}', ...)
// Controller: public function show(CorpoCeleste $corpoCeleste)
// URL: /corpi-celesti/terra  risolve automaticamente il modello
```

Se non trova il modello 404 automatico. `findOrFail()` è implicito.

API Route

Route che restituisce JSON invece di HTML. Definita in `routes/api.php`, automaticamente prefissata con `/api`. Pubbliche (nessuna auth) o prottte con middleware. Controller dedicato in `app/Http/Controllers/Api/`. API Resource per trasformare il modello in JSON controllato.

Job

Classe che esegue un task in background (queue). Utile per operazioni lunghe (import, email, processing). `ShouldBeUnique` evita duplicati. Dispatch: `ImportNasalmage::dispatch($url)`. Gestione errori con `failed()`. In Astralis: `ImportNasalmage` importa immagini NASA. `php artisan make:job`.

Route Groups

Raggruppamento di route con prefisso, middleware o namespace condivisi. Riduce ripetizioni.

```
Route::prefix('admin')->middleware('auth')->group(function () {  
    Route::get('/dashboard', [DashboardController::class, 'index']);  
    Route::resource('categorie', CategoriaController::class);  
});
```

`prefix('admin')` aggiunge `/admin` a tutte le route nel gruppo. `middleware('auth')` applica il middleware a tutte. In Astralis: route admin raggruppate in `routes/web.php`.

Service Provider

Classe che registra servizi nel container di Laravel. Il punto di bootstrap dell'applicazione. `register()` registra i servizi, `boot()` esegue logica dopo che tutti i provider sono registrati.

```
// app/Providers/AppServiceProvider.php  
public function boot()  
{  
    // Logica che gira dopo tutti i provider  
}
```

In Laravel 11+: `bootstrap/providers.php` elenca i provider. Il provider è dove registri binding custom, event listener, view composer, policy. In Astralis: `AuthServiceProvider` registra policy e gate.

Service Container (IoC)

Il cuore di Laravel. Container che gestisce la creazione e l'iniezione delle dipendenze. Quando scrivi `public function __construct(NasalmageService $service)`, il Container crea automaticamente l'istanza di `NasalmageService` e la inietta.

```
// Binding manuale (raramente necessario)
app()->bind('slack', fn() => new SlackClient(config('slack.token')));
$slack = app('slack'); // Risolve dal container

// Dependency Injection — il modo standard
class CorpoCelesteController extends Controller {
    public function __construct(
        protected NasalmageService $service, // Iniettato automaticamente
    ) {}
}

// app() — accede al container
$user = app('auth')->user();
```

Il Container risolve le dipendenze automaticamente tramite **reflection**. Se una classe ha bisogno di altre classi, il Container le crea tutte. `bind()` per registrazioni custom, `singleton()` per istanze condivise. Il 90% delle volte non serve toccarlo direttamente — Laravel lo fa da solo.

Eloquent Scopes

Metodi del Model che incapsulano query ricorrenti. Prefisso `scope` + nome camelCase. Accessibili come metodi del modello.

```
// Nel Model CorpoCeleste
public function scopeInEvidenza($query)
{
    return $query->where('in_evidenza', true);
}

public function scopeByCategoria($query, $categoricalId)
{
    return $query->where('categoria_id', $categoricalId);
}

// Uso
CorpoCeleste::inEvidenza()->get();
CorpoCeleste::byCategoria(1)->orderBy('nome')->get();
```

Scopes rendono le query leggibili e riusabili. In Astralis: `scopeInEvidenza` per la homepage, `scopeByCategoria` per il catalogo filtrato.

Soft Deletes

Trait che “cancella morbido”: invece di eliminare la riga dal DB, imposta `deleted_at` con timestamp. La riga resta nel DB ma non appare nelle query normali.

```
// Migration — aggiunge colonna deleted_at
$table->softDeletes();

// Model — usa il trait
class Missione extends Model {
    use SoftDeletes;
}

// Operazioni
$missione->delete();      // Imposta deleted_at (soft delete)
$missione->forceDelete();  // Elimina davvero dal DB
Missione::withTrashed()->get(); // Include le righe cancellate
Missione::onlyTrashed()->get(); // Solo le righe cancellate
$missione->restore();      // Ripristina (azzerà deleted_at)
```

Astralis usa **hard delete** (eliminazione diretta con `cascadeOnDelete`) perché il progetto non necessita di recovery. Soft deletes sono lo standard per dati critici (utenti, ordini, fatture).

Autenticazione Breeze

Cos'è: Laravel Breeze è il package ufficiale di autenticazione. Fornisce login, register, profilo, reset password, verifica email — tutto pronto.

Installazione:

```
composer require laravel/breeze --dev
php artisan breeze:install blade
npm install && npm run build
```

Route generate (`routes/auth.php`):

Route	Metodo	Cosa fa
<code>/login</code>	GET/POST	Form + login
<code>/register</code>	GET/POST	Form + registrazione

/logout	POST	Logout
/forgot-password	GET/POST	Reset password via email
/reset-password/{token}	GET/POST	Form reset con token
/email/verify	GET	Verifica email
/profile	GET/PUT	Modifica profilo

Middleware:

- `auth` verifica login, redirect a `/login` se non loggato
- `guest` solo utenti non loggati (es. pagina login)
- `verified` richiede email verificata

Astralis: Breeze Blade puro (non Inertia, rimosso). Route admin protette da middleware `auth`.
Tema dark `#0A0A1A`. Admin demo: `admin@astralis.it` / `password`.

CRUD — Resource Controller

Cos'è: Resource Controller è un controller con **7 metodi standard** che gestisce tutte le operazioni CRUD su un'entità.

I 7 metodi:

#	Metodo	HTTP	Cosa fa
1	<code>index()</code>	GET	Lista tutti gli elementi
2	<code>create()</code>	GET	Mostra form di creazione
3	<code>store()</code>	POST	Salva il nuovo elemento
4	<code>show(\$id)</code>	GET	Mostra dettaglio elemento
5	<code>edit(\$id)</code>	GET	Mostra form di modifica
6	<code>update(\$id)</code>	PUT/PATCH	Aggiorna l'elemento
7	<code>destroy(\$id)</code>	DELETE	Elimina l'elemento

Comandi:

```
# Crea Resource Controller
php artisan make:controller Admin/CategoriaController --resource
```

```
# Route resource (genera tutte le route CRUD)
Route::resource('categorie', CategoriaController::class);
```

Route generate (12 totali):

Nome route	HTTP	URI	Metodo
categorie.index	GET	/categorie	index
categorie.create	GET	/categorie/create	create
categorie.store	POST	/categorie	store
categorie.show	GET	/categorie/{id}	show
categorie.edit	GET	/categorie/{id}/edit	edit
categorie.update	PUT/PATCH	/categorie/{id}	update
categorie.destroy	DELETE	/categorie/{id}	destroy

Esempio ridotto (senza create/edit, come in Astralis):

```
Route::resource('categorie', CategoriaController::class)->except(['create', 'edit']);
```

Astralis: 8 controller admin, 13 FormRequest (Store/Update per ogni entità). Autorizzazione con Policy `before()` — admin bypassa tutto. Form con Alpine.js per interattività.

Upload Media

Cos'è: Meccanica per caricare file (immagini, copertine, logo) dal form al server.

Installazione:

```
composer require intervention/image
```

Configurazione storage:

```
php artisan storage:link # Crea symlink: public/storage storage/app/public
```

Nel controller:

```
// Salva il file nel disco 'public' nella cartella 'corpi'
$path = $request->file('immagine')->store('corpi', 'public');
// Risultato: "corpi/abc123.jpg"
```

```
// Percorso fisico: storage/app/public/corpi/abc123.jpg
// URL web: /storage/corpi/abc123.jpg
```

Resize con Intervention Image v4:

```
use Intervention\Image\ImageManager;
use Intervention\Image\Drivers\Gd\Driver;

$manager = new ImageManager(new Driver());
$image = $manager->decodePath($request->file('immagine'));
$image->scaleDown(1200, 1200); // Preserva aspect ratio
$image->save(storage_path('app/public/' . $path));
```

Attenzione: In Intervention Image v4, `Image::read()` è stato rinominato in `decodePath()` / `decodeBinary()`. La facade è stata rimossa.

ImageUploadService (Astralis):

Service Layer che centralizza upload + resize + validazione. Riusabile da controller admin, observer, CLI.

```
// Nel controller
$this->uploadService->upload($request->file('immagine'), 'corpi');
```

Astralis — tipi di upload:

Entità	Cartella	Dimensione	Strumento
Missioni	missioni/	Logo 300px	ImageUploadService
Galleria	galleria/	1200px	ImageUploadService
CorpiCelesti	corpi/	Copertina	ImageUploadService
Galleria NASA	—	URL remoto	Niente download locale

Relazioni Eloquent

Le relazioni definiscono come i modelli Eloquent si collegano tra loro nel database. Laravel le dichiara come metodi nel modello e le risolve automaticamente nelle query.

Tabella riassuntiva:

Tipo	Direzione	Metodo Eloquent	Tabella pivot	Esempio generico
------	-----------	-----------------	---------------	------------------

HasOne	1 1	hasOne()	No	User Profile
BelongsTo	N 1	belongsTo()	No	Post User
HasMany	1 N	hasMany()	No	User Post
BelongsToMany	N N	belongsToMany()	Sì	Post Tag
HasManyThrough	1 N N	hasManyThrough()	No	Country Posts (through Users)

Cos'è una Foreign Key (FK)

Una **Foreign Key** è una colonna che collega una tabella a un'altra. "Ricorda" l'ID di un'altra tabella.

Analogia: un'etichetta sullo scrigno che dice "questo scrigno appartiene alla categoria X".

```
// Nella migration della tabella figlia (corpi_celesti)
$table->foreignId('categoria_id')->constrained('categorie')->cascadeOnDelete();
```

Pezzo	Cosa fa
foreignId('categoria_id')	Crea la colonna categoria_id (unsigned big integer)
->constrained('categorie')	Vincola: il valore DEVE esistere nella tabella categorie
->cascadeOnDelete()	Se elimini la categoria elimina anche tutti i corpi celesti figli

Senza FK: puoi inserire categoria_id = 999 anche se la categoria 999 non esiste dati orfani.

Con FK: il database blocca l'inserimento se la categoria 999 non esiste integrità referenziale.

HasOne (1 1)

Una riga collegata esiste per ogni riga del modello. Esempio: ogni User ha un Profile.

```
// Nel modello User
public function profile(): HasOne
{
    return $this->hasOne(Profile::class);
}
// $user->profile singolo oggetto Profile
```

Non usato in Astralis: nessuna relazione 1 1 necessaria tra le entità.

BelongsTo (N 1)

Ogni riga del modello appartiene a una riga di un altro modello. È il lato **“figlio”** della relazione 1-N.

```
// Nel modello CorpoCeleste
public function categoria(): BelongsTo
{
    return $this->belongsTo(Categoria::class);
}
// $corpo->categoria    singola Categoria
```

Usato in Astralis per: CorpoCeleste Categoria, GalleriaCorpo CorpoCeleste, Curiosità CorpoCeleste.

HasMany (1 N)

Una riga del modello è collegata a più righe di un altro modello. È il lato **“padre”** della relazione 1-N.

```
// Nel modello Categoria
public function corpiCelesti(): HasMany
{
    return $this->hasMany(CorpoCeleste::class);
}
// $categoria->corpiCelesti    Collection di CorpoCeleste
```

Usato in Astralis per: Categoria CorpiCeleste, CorpoCeleste Galleria, CorpoCeleste Curiosità.

Regola: se `CorpoCeleste` ha `belongsTo(Categoria)` , allora `Categoria` ha `hasMany(CorpoCeleste)` . Sono le **due facce della stessa medaglia**.

BelongsToMany (N N)

Molti modelli sono collegati a molti altri. Richiede una **tabella pivot** (tabella intermedia) che memorizza le foreign key di entrambi. La pivot può avere colonne aggiuntive.

```
// Nel modello CorpoCeleste
public function missioni(): BelongsToMany
{
    return $this->belongsToMany(Missione::class, 'corpo_celeste_missione')
        ->withPivot('tipo_esplorazione', 'anno_arrivo');
}
```

```
// $corpo->missioni    Collection di Missione
// $corpo->missioni->first()->pivot->tipo_esplorazione
```

Tabella pivot corpo_celeste_missione :

colonna	tipo	Note
id	bigIncrements	PK auto-incrementale
corpo_celeste_id	FK corpi_celesti	cascadeOnDelete
missione_id	FK missioni	cascadeOnDelete
tipo_esplorazione	string(50), nullable	dati aggiuntivi
anno_arrivo	year, nullable	dati aggiuntivi
unique	[corpo_celeste_id, missione_id]	no duplicati

Cos'è ->withPivot() ? Le colonne aggiuntive della pivot NON vengono caricate di default.
withPivot('colonna') le include nella relazione.

Usato in Astralis per: CorpoCeleste Missione. Scelta giustificata: un corpo celeste può essere oggetto di più missioni (es. Voyager ha visitato più pianeti), e una missione può esplorare più corpi.

HasManyThrough (1 N N)

Una relazione indiretta: il modello A è collegato a molteplici modello C attraverso il modello B (che ha relazione 1-N con C).

```
// Nel modello Country
public function posts(): HasManyThrough
{
    return $this->hasManyThrough(Post::class, User::class);
}
// $country->posts    Collection di Post (attraverso User)
```

Non usato in Astralis: le relazioni sono sufficientemente dirette senza bisogno di attraversamento.

Come creare una relazione — Comandi Artisan

Workflow relazione 1-N (es. Categoria CorpoCeleste)

```
# 1. Crea il modello PADRE + migration
php artisan make:model Categoria -m

# 2. Crea il modello FIGLIO + migration
php artisan make:model CorpoCeleste -m
```

Nella **migration del figlio** (corpi_celesti), aggiungi la FK:

```
$table->foreignId('categoria_id')
    ->constrained('categorie')
    ->cascadeOnDelete();
```

Nei **modelli**, definisci entrambe le facce:

```
// PADRE (Categoria) — lato "ha molti"
public function corpiCelesti(): HasMany
{
    return $this->hasMany(CorpoCeleste::class);
}

// FIGLIO (CorpoCeleste) — lato "appartiene a"
public function categoria(): BelongsTo
{
    return $this->belongsTo(Categoria::class);
}
```

Workflow relazione N-N (es. CorpoCeleste Missione)

```
# 1. Crea i due modelli
php artisan make:model CorpoCeleste -m
php artisan make:model Missione -m

# 2. Crea la migration PIVOT (tabella intermedia)
php artisan make:migration create_corpo_celeste_missione_table
```

Nella **migration pivot**:

```
Schema::create('corpo_celeste_missione', function (Blueprint $table) {
    $table->id();
    $table->foreignId('corpo_celeste_id')->constrained('corpi_celesti')->cascadeOnDelete();
    $table->foreignId('missione_id')->constrained('missioni')->cascadeOnDelete();
    $table->string('tipo_esplorazione', 50)->nullable(); // dati aggiuntivi
});
```

```
$table->year('anno_arrivo')->nullable();          // dati aggiuntivi
$table->unique(['corpo_celeste_id', 'missione_id']); // no duplicati
$table->timestamps();
});
```

Nei **modelli** (entrambi):

```
// CorpoCeleste
public function missioni(): BelongsToMany
{
    return $this->belongsTo(Missione::class, 'corpo_celeste_missione')
        ->withPivot('tipo_esplorazione', 'anno_arrivo');
}

// Missione
public function corpiCelesti(): BelongsToMany
{
    return $this->belongsTo(CorpoCeleste::class, 'corpo_celeste_missione')
        ->withPivot('tipo_esplorazione', 'anno_arrivo');
}
```

Nota: il nome della tabella pivot segue la convenzione:

nomeplurale_modello1_nomeplurale_modello2 in ordine alfabetico (corpo_celeste_missione ,
non missione_corpo_celeste).

Come usare le relazioni nelle query

```
// Eager loading (evita problema N+1)
$corpi = CorpoCeleste::with('categoria')->get(); // 2 query, non 1+N

// Accesso diretto
$nomeCategoria = $corpo->categoria->nome; // "Pianeta"

// Filtraggio per relazione
$corpi = CorpoCeleste::whereHas('categoria', fn($q) => $q->where('nome', 'Pianeta'))->get();

// Conteggio relazioni
$categorie = Categoria::withCount('corpiCelesti')->get();
// Ogni categoria avrà: $categoria->corpi_celesti_count

// Eager loading multipli
$corpo = CorpoCeleste::with(['categoria', 'galleria', 'curiosita', 'missioni'])->first();
```

Problema N+1: quando in un loop accedi a una relazione senza eager loading.

```
// SBAGLIATO: 10 corpi × 1 categoria = 11 query
$corpi = CorpoCeleste::all();
foreach ($corpi as $corpo) {
    echo $corpo->categoria->nome; // Query separata ogni volta!
}

// CORRETTO: 2 query totali
$corpi = CorpoCeleste::with('categoria')->get();
foreach ($corpi as $corpo) {
    echo $corpo->categoria->nome; // Già caricato, zero query extra
}
```

Perché queste relazioni — riferimento alla traccia

La traccia richiede:

- “Deve esserci almeno una seconda entità collegata alla prima con relazione 1-N o N-N.”
([progetto-finale.md:30](#))
- “Più relazioni implementate, più completo sarà il vostro gestionale!” ([progetto-finale.md:43](#))

Requisito traccia	Realizzato in Astralis
“almeno una 1-N”	3 relazioni 1-N: Categoria CorpoCeleste, CorpoCeleste Galleria, CorpoCeleste Curiosità
“o N-N”	1 relazione N-N: CorpoCeleste Missione (con pivot corpo_celeste_missione)
“CRUD per seconda entità”	4 CRUD secondari: Categoria, Missione, Curiosità, Galleria — tutte con CRUD completo

Domande esame tipiche

Q: Cos’è una Foreign Key?

È una colonna che collega una tabella a un’altra. Esempio: categoria_id nella tabella corpi_celesti punta all’ID della tabella categorie . Il database verifica che il valore esista. Senza FK puoi inserire ID inesistenti.

Q: Qual è la differenza tra HasMany e BelongsTo?

Sono le due facce della stessa relazione 1-N. HasMany è sul lato padre (Categoria ha molti corpi). BelongsTo è sul lato figlio (CorpoCeleste appartiene a una categoria). Ogni relazione 1-N richiede entrambi.

Q: Cos'è una tabella pivot?

Tabella intermedia che risolve una relazione N-N. Contiene solo le FK dei due modelli + eventuali colonne aggiuntive. Esempio: `corpo_celeste_missione` collega `CorpoCeleste` e `Missione` con `tipo_esplorazione` e `anno_arrivo`.

Q: Quando usare `cascadeOnDelete`?

Quando elimina il padre deve eliminare anche i figli. In Astralis: `categoria_id->constrained()->cascadeOnDelete()` elimino "Pianeta" elimino anche tutti i corpi celesti di quella categoria. Utile per pulizia automatica.

Q: Cos'è il problema N+1 e come lo risolvi?

Quando in un loop accedi a una relazione senza eager loading: N query extra. Soluzione: `CorpoCeleste::with('categoria')` carica tutto in anticipo con 2 query invece di 1+N.

Q: Perché N-N per missioni e non 1-N?

Perché una missione spaziale può esplorare più corpi celesti (Voyager 1 ha visitato Giove, Saturno, Urano, Nettuno) E un corpo celeste può essere stato esplorato da più missioni (Marte: Curiosity, Perseverance, InSight...). La tabella pivot memorizza anche dati extra: `tipo_esplorazione` e `anno_arrivo`.

Eager Loading

Carica le relazioni in anticipo per evitare N+1 query. `CorpoCeleste::with(['categoria', 'galleria'])` fa 2 query invece di 1 + N.

API Resource

Trasforma un modello in JSON. `CorpoCelesteResource` seleziona campi da esporre. Espone `nome` (italiano) per il frontend guest. Protegge dati interni. `php artisan make:resource`.

API e REST — Definizioni e Implementazione

Cos'è un'API

API = Application Programming Interface. È un contratto che definisce come due software comunicano tra loro.

Analogia: un cameriere in ristorante.

- Il **cliente** (React frontend) fa un ordine: "Voglio la lista dei pianeti"
- Il **cameriere** (API) porta la richiesta alla cucina (Laravel backend)
- La **cucina** prepara il piatto (query al database)
- Il cameriere **restituisce** il piatto (risposta JSON)

```
// React (cliente) chiama l'API
const response = await fetch("/api/corpi-celesti");
```

```
const data = await response.json(); // JSON con 18 corpi celesti
```

```
// Laravel (cucina) restituisce i dati
public function index(): JsonResponse
{
    return CorpoCelesteResource::collection(CorpoCeleste::all());
}
```

In Astralis: l'API è il **ponte** tra il React guest (SPA standalone) e il database MySQL.

API vs Route — Qual è la differenza?

	Route web (Blade)	API (JSON)
Cosa restituisce	HTML (pagina intera)	JSON (solo dati)
Chi la usa	Browser umano	JavaScript (React, fetch, axios)
Autenticazione	Session + cookie (Breeze)	API pubbliche (o token)
Dove in Laravel	routes/web.php	routes/api.php
Template	Blade (@extends , @section)	Nessuno (JSON puro)
Esempio Astralis	/admin/corpi-celesti vista Blade	/api/corpi-celesti JSON

```
// Route web    restituisce HTML (Blade)
Route::get('/admin/corpi-celesti', [CorpoCelesteController::class, 'index'])
    ->name('admin.corpi-celesti.index');

// Route API    restituisce JSON
Route::get('/corpi-celesti', [CorpoCelesteController::class, 'index']);
```

Regola pratica: se il consumatore è un browser umano → route web. Se il consumatore è del codice JavaScript → API.

Cos'è lo standard REST?

REST = REpresentational State Transfer. Non è un protocollo, è un **stile architetturale** per progettare API web. Definito da Roy Fielding nel 2000.

6 vincoli (in parole semplici):

#	Vincolo	Cosa significa	In Astralis
---	---------	----------------	-------------

1	Client-Server	Frontend e backend sono separati	React (client) Laravel (server)
2	Stateless	Ogni richiesta è indipendente	API senza sessione; ogni GET è autoconsistente
3	Cacheable	Le risposte possono essere cachate	Cache su stats (1h) e corpi-celesti (5min)
4	Layered System	Il client non sa cosa c'è tra lui e il server	Vite proxy inoltra /api a localhost:8000
5	Uniform Interface	URL = risorse, HTTP methods = azioni	/api/corpi-celesti è sempre una risorsa
6	Code on Demand (opzionale)	Il server può inviare codice eseguibile	Non usato in Astralis

In pratica: REST dice che le URL devono rappresentare **risorse** (nomi, non verbi) e gli HTTP methods devono rappresentare **azioni**.

REST: GET /api/corpi-celesti (leggi tutte le risorse)
 REST: GET /api/corpi-celesti/terra (leggi una risorsa specifica)
 Non REST: GET /api/getCorpiCelesti (URL contiene un verbo)
 Non REST: POST /api/deleteCorpo/5 (DELETE non dovrebbe essere POST)

Metodi HTTP

Metodo	Cosa fa	Astralis	Status codes
GET	Legge una risorsa	10 endpoint API pubblici	200 (ok), 404 (non trovato)
POST	Crea una risorsa	Solo admin (autenticato)	201 (creato), 422 (validazione fallita)
PUT/PATCH	Aggiorna una risorsa	Solo admin (autenticato)	200 (aggiornato), 422 (validazione fallita)
DELETE	Elimina una risorsa	Solo admin (autenticato)	204 (eliminato), 404 (non trovato)

Status codes più importanti:

Code	Significato	Quando in Astralis
------	-------------	--------------------

200 OK	Richiesta riuscita	Tutte le GET che restituiscono dati
201 Created	Risorsa creata	<code>store()</code> admin — crea un corpo celeste
204 No Content	Eliminata con successo	<code>destroy()</code> admin — elimina una curiosità
404 Not Found	Risorsa inesistente	Slug non trovato (Route Model Binding)
403 Forbidden	Accesso negato	Non-admin tenta CRUD (Policy before() false)
422 Unprocessable Entity	Dati di validazione errati	FormRequest fallita (campi obbligatori mancanti)
429 Too Many Requests	Throttle exceeded	>60 req/min API, >120 req/min admin
500 Server Error	Errore interno	Bug, cache rotta, DB down

Cosa sono gli API Resource?

Un **API Resource** è una classe che trasforma un modello Eloquent in JSON controllato. Decide **quali campi** esporre e **quali nascondere**.

Perché serve: il modello Eloquent ha tutti i campi (inclusi dati interni). L'API Resource seleziona solo ciò che il frontend ha bisogno.

```
// Senza Resource   esponi TUTTO (inclusi dati interni)
return response()->json(CorpoCeleste::find(1));

// Con Resource   controlli cosa esporre
return new CorpoCelesteResource(CorpoCeleste::find(1));
```

Esempio concreto: `CorpoCelesteResource` espone `nome` (italiano) ma **nasconde** `immagine_utente` (boolean, interno al sistema).

Creare un API Resource

```
php artisan make:resource CorpoCelesteResource
```

Crea il file `app/Http/Resources/CorpoCelesteResource.php` :

```
class CorpoCelesteResource extends JsonResource
{
    public function toArray(Request $request): array
    {
```

```

return [
    'id' => $this->id,
    'nome' => $this->nome,      // Esposto: il frontend lo usa
    'slug' => $this->slug,
    'categoria' => new CategoriaResource($this->whenLoaded('categoria')),
    'immagine_url' => $this->immagine_url,
    // 'immagine_utente' => ...    // NON esposto: è un dato interno
];
}
}

```

Nel controller:

```

public function show(CorpoCeleste $corpoCeleste): CorpoCelesteResource
{
    $corpoCeleste->load(['categoria', 'galleria', 'curiosita', 'missioni']);
    return new CorpoCelesteResource($corpoCeleste);
}

```

5 API Resource in Astralis:

Resource	Modello	Note
CorpoCelesteResource	CorpoCeleste	Adaptive: cambia output in base a <code>?detail=true</code>
CategoriaResource	Categoria	Include <code>corpi_count</code> quando caricato
MissioneResource	Missione	Smart URL: risolve remote vs locale + conditional pivot
CuriositaResource	Curiosita	Include corpo celeste padre
GalleriaCorpoResource	GalleriaCorpo	Accessor <code>percorso_url</code> per URL remote/locali

Come Astralis implementa REST

10 endpoint API, tutti GET, tutti pubblici:

#	Route	Metodo	Cosa restituisce	Filtri
1	<code>/api/corpi-celesti</code>	GET	Lista corpi (paginata)	<code>?categoria</code> , <code>?tipo</code> , <code>?search</code> , <code>?in_evidenza</code> , <code>?per_page</code>
2	<code>/api/corpi-celesti/{slug}</code>	GET	Dettaglio corpo	Eager load: categoria, galleria, curiosità, missioni

3	/api/corpi-celesti/{slug}/simili	GET	Max 4 corpi simili	Stessa categoria, esclude quello corrente
4	/api/categorie	GET	Tutte le categorie	Nessun filtro
5	/api/categorie/{slug}	GET	Dettaglio categoria + corpi	Eager load: corpiCelesti con galleria
6	/api/missioni	GET	Lista missioni (paginata)	?agenzia , ?stato
7	/api/missioni/{slug}	GET	Dettaglio missione + corpi	Eager load: corpiCelesti con categoria e galleria
8	/api/curiosita	GET	Lista curiosità (paginata)	Nessun filtro
9	/api/galleria	GET	Lista galleria (paginata)	Ordinamento per ordine + created_at
10	/api/dashboard/stats	GET	Statistiche generali	Cache 1 ora, raw SQL aggregation

Pattern avanzati implementati:

Pattern	Come funziona	Perché
Route Model Binding (slug)	{corpoCeleste:slug} invece di {id}	URL leggibili, SEO-friendly
Adaptive Resource	CorpoCelesteResource cambia output (list vs detail)	Stessa route, meno dati in lista, tutto in dettaglio
Cache ID-based	Cachea solo gli ID (pluck('id')), re-query con whereIn()	Evita serializzazione rotta di Collection
Throttle	throttle:60,1 su API, throttle:120,1 su admin	Protezione da abuso
Eager Loading	with(['categoria', 'galleria'])	Previene problema N+1
per_page bounds	max(1, min(\$request->integer('per_page', 12), 100))	Default 12 (corpi) o 20 (altri), max 100

REST puro vs Astralis — dove si discosta e perché

REST puro	Astralis	Perché la deviazione
-----------	----------	----------------------

CRUD con GET/POST/PUT/DELETE su API	Solo GET pubbliche, POST/PUT/DELETE in admin Blade	Il guest non deve creare/modificare/eliminare
Auth su tutte le API	API completamente pubbliche	Il visitatore deve poter esplorare senza login
OAuth/JWT per auth API	Session + Breeze solo per admin	Breeze è sufficiente per un progetto esame
Cache su tutte le GET	Cache solo su 2 endpoint (stats + corpi)	Gli altri endpoint hanno poco traffico
HATEOAS (link ipertestuali)	No	Complessità non necessaria per l'esame
Versioning (/api/v1/)	No	Un solo versione, non serve

Nota per l'esame: queste deviazioni sono **scelte progettuali deliberate**, non errori. Spiegarle dimostra comprensione dello standard REST e capacità di adattarlo al contesto reale.

Domande esame tipiche

Q: Cos'è un'API?

Application Programming Interface. È un contratto che definisce come due software comunicano. In Astralis: React (client) chiama Laravel (server) tramite endpoint JSON.

Q: Qual è la differenza tra route web e API?

Route web restituisce HTML (pagine Blade) per browser umani. API restituisce JSON per codice JavaScript. Route web: `routes/web.php` . API: `routes/api.php` .

Q: Cos'è REST?

Representational State Transfer. Stile architetturale con 6 vincoli: client-server, stateless, cacheable, layered system, uniform interface, code on demand. In pratica: URL = risorse, methods = azioni, JSON = formato.

Q: Perché solo GET nelle API pubbliche?

Il guest non deve creare, modificare o eliminare dati. Le operazioni CRUD sono solo nell'admin autenticato. Le API pubbliche servono solo a leggere i dati per il frontend React.

Q: Come gestisci la paginazione?

`LengthAwarePaginator` manuale (corpi-celesti) o `->paginate()` (altri). Default 12-20, max 100. Parametri: `?page=N&per_page=N` .

Q: Perché lo slug invece dell'ID nelle URL?

URL leggibili e SEO-friendly: `/api/corpi-celesti/terra` invece di `/api/corpi-celesti/3` . Laravel risolve automaticamente con Route Model Binding: `{corpoCeleste:slug}` .

9. Definizioni React

Componente

Unità base di React. Function component: `function Card({ title }) { return <h1>{title}</h1> }` .
Ogni componente è un pezzo di UI riutilizzabile.

JSX

Syntax extension che compila in `React.createElement()` . Permette di scrivere HTML-like nel JavaScript. `const el = <h1>Ciao</h1>` `React.createElement('h1', null, 'Ciao')` .

Hook

Funzioni che aggiungono state/lifecycle ai componenti. Regole: (1) solo nel top level, (2) mai dentro if/cicli, (3) solo in componenti React.

useState

Hook per state locale. `const [count, setCount] = useState(0)` . Ogni `setState` triggera re-render.

useReducer

Hook per state complessi con logica deterministica. Alternativa a `useState` quando lo state ha molti sub-campi o transizioni complesse.

```
const [state, dispatch] = useReducer(reducer, initialState);

function reducer(state, action) {
  switch (action.type) {
    case "SET_DATA":
      return { ...state, data: action.payload, loading: false };
    case "SET_ERROR":
      return { ...state, error: action.payload, loading: false };
    case "SET_LOADING":
      return { ...state, loading: true };
    default:
      return state;
  }
}

// Uso
dispatch({ type: "SET_DATA", payload: corpiCelesti });
```

Differenza da `useState` : la logica di transizione è centralizzata nel `reducer` (funzione pura: `state + action => new state`). Più testabile, più prevedibile per state complessi. In Astralis: `useFetch.js` usa `useReducer` per gestire loading/error/data con dispatch `SET_LOADING` , `SET_DATA` , `SET_ERROR` .

useEffect

Hook per side effects. `useEffect(() => { fetchData(); return () => controller.abort(); }, [deps])` .
Dipendenze: quando cambiano, riesegue. Return: cleanup.

Props

Dati passati da padre a figlio. `<Card title="Terra" />` il figlio riceve `{ title: "Terra" }` . Read-only.

Key prop

Prop speciale nelle liste. `key={item.id}` identifica univocamente ogni elemento. Aiuta React a capire quali elementi sono stati aggiunti/rimossi/riordinati. Senza key, React rirenderizza tutta la lista.

```
{
  corpi.map((corpo) => <CorpoCard key={corpo.id} corpo={corpo} />);
}
```

Usa sempre ID univoci come key, mai l'indice dell'array (`key={index}` problemi con riordinamento).

Virtual DOM

Copia leggera del DOM reale. React calcola la differenza (diffing) e aggiorna solo le parti cambiate del DOM reale. Più performante che manipolare il DOM direttamente.

Custom Hook

Funzione che inizia con `use` e usa altri hook. `useFetch(url)` usa `useState` + `useEffect` + `AbortController` . Riutilizzabile tra componenti.

Error Boundary

Class component con `componentDidCatch()` . Cattura errori nei figli. Non cattura errori in event handlers o codice async. In Astralis: in `App.jsx` avvolge `<Routes>` .

React.memo()

`React.memo(Component)` il componente non re-renderizza se le props non cambiano. Ottimizzazione per componenti costosi. Differenza: `useMemo()` memoizza un valore, `useCallback()` memoizza una funzione.

useMemo

Hook che memorizza il **risultato di un calcolo** tra i render. Solo ricalcola quando le dipendenze cambiano.

```
// Senza useMemo — ricalcola OGNI render (anche se i dati non cambiano)
const slides = galleria.map((img) => ({ src: img.percorso }));

// Con useMemo — ricalcola SOLO quando galleria cambia
const slides = useMemo(
  () => galleria.map((img) => ({ src: img.percorso })),
  [galleria],
);
```

Quando usarlo: calcoli costosi, trasformazioni di array grandi, valori che passi a child memoizzati. In Astralis: `SolarSystem.jsx` memoizza la posizione delle stelle, `LightboxGalleria.jsx` memoizza gli slides, `HomePage.jsx` memoizza lo sfondo stellare.

useCallback

Hook che memorizza una **funzione** tra i render. Evita che la funzione venga ricreata ad ogni render (utile quando la passi a child memoizzati).

```
// Senza useCallback — nuova funzione OGNI render il child re-renderizza
const handleClick = (id) => {
  setSelected(id);
};

// Con useCallback — stessa funzione finché deps non cambiano
const handleClick = useCallback((id) => {
  setSelected(id);
}, []);
```

Differenza da `useMemo` : `useMemo(fn)` memorizza il **risultato** di `fn()` , `useCallback(fn)` memorizza la **funzione stessa**. In Astralis: `useInView.js` usa `useCallback` per il ref callback, `Navbar.jsx` per la funzione di chiusura mobile menu.

Conditional rendering

Mostrare/nascondere elementi in base a condizioni. 3 pattern principali:

```
// 1. && (and) — mostra se vero
{
  isLoading && <CorpoCard />;
}

// 2. Ternario — mostra A o B
{
  error ? <Error message={error} /> : <CorpoCard />;
}

// 3. Early return — ritorna nulla se condizione non soddisfatta
if (!corpo) return null;
return <CorpoCard corpo={corpo} />;
```

In React non puoi usare `if/else` direttamente nel JSX (solo espressioni). Il ternario e `&&` sono i pattern più comuni.

Suspense

Componente che mostra un **fallback** (caricamento) mentre i figli sono in attesa di dati lenti (lazy loading, data fetching).

```
import { Suspense, lazy } from "react";

const HomePage = lazy(() => import("./pages/HomePage"));

function App() {
  return (
    <Suspense fallback=<PageLoader />>
      <Routes>
        <Route path="/" element=<HomePage /> />
      </Routes>
    </Suspense>
  );
}
```

Il fallback viene mostrato finché tutti i figli non sono “pronti”. Se un figlio lancia un’eccezione, l’Error Boundary cattura. In Astralis: `<Suspense fallback=<PageLoader />>` wrappa tutte le Routes in `App.jsx` per gestire il caricamento delle pagine lazy-loaded.

React.lazy / Code Splitting

Caricare componenti **solo quando servono** invece di tutto in un file JS enorme.

```
// Senza lazy — tutto caricato subito (bundle grande)
import HomePage from "../pages/HomePage";
import CorpiLista from "../pages/CorpiLista";

// Con lazy — caricato al bisogno (bundle diviso)
const HomePage = lazy(() => import("../pages/HomePage"));
const CorpiLista = lazy(() => import("../pages/CorpiLista"));
```

`lazy()` accetta una funzione che restituisce un `import()` dinamico. Vite crea un file JS separato per ogni lazy component. Quando l'utente naviga a `/corpi-celesti`, il browser scarica solo il JS di `CorpiLista`. In Astralis: tutte le 5 pagine sono lazy-loaded in `App.jsx`. Il code splitting riduce il bundle iniziale da ~200KB a ~50KB.

AbortController

Cancella fetch in corso. `const controller = new AbortController()` `fetch(url, { signal: controller.signal })` `controller.abort()` nel cleanup di `useEffect`. Evita memory leak.

requestAnimationFrame

API del browser per animazioni fluenti. `callback` viene chiamato prima del repaint del browser. In Astralis: orbite del sistema solare. Più performante di `setInterval` perché si sincronizza con il refresh rate del display.

Axios

Libreria HTTP per JavaScript. Semplifica le chiamate API rispetto a `fetch()` nativo: gestisce automaticamente il parsing JSON, gli errori HTTP, i timeout, e gli interceptor. In Astralis: `apiClient.js` è un'istanza axios con retry + timeout + proxy. Installata con `npm install axios`.

```
// fetch() nativo — verbose, errori gestiti manualmente
const res = await fetch("/api/corpi-celesti");
if (!res.ok) throw new Error(res.status); // bisogna controllare sempre
const data = await res.json(); // parsing manuale

// axios — conciso, errori gestiti dall'interceptor
const { data } = await apiClient.get("/corpi-celesti");
// errori HTTP (4xx, 5xx) vengono gestiti dall'interceptor in apiClient.js
```

Caratteristica	fetch() nativo	axios
----------------	----------------	-------

Parsing JSON	Manuale (<code>res.json()</code>)	Automatico (<code>response.data</code>)
Errori HTTP	Solo network error (deve controllare <code>res.ok</code>)	Tutti gli errori (4xx, 5xx)
Timeout	Bisogna implementare con <code>AbortController</code>	Opzione nativa (<code>timeout: 30000</code>)
Interceptor	Nessuno	Sì (gestione centralizzata errori)
Retry	Bisogna implementare	Sì (nell'interceptor)

useRef

Hook che crea un riferimento mutabile. Utile per: accedere a un elemento DOM, persistere valori tra render senza triggerare re-render, salvare timer/interval.

```
const inputRef = useRef(null);
const prevValue = useRef("");

// Accesso DOM
<input ref={inputRef} />;
inputRef.current.focus();

// Persistenza senza re-render
useEffect(() => {
  prevValue.current = newValue;
}, [newValue]);
```

`useRef` restituisce `{ current: }`. A differenza di `useState`, modificare `.current` **non** causa re-render. In Astralis: usato in `SolarSystem.jsx` (`angleRef`, `lastTimeRef` per persistere l'angolo di animazione tra frame), `useInView.js` (`optionsRef`, `observerRef` per salvare l'istanza `IntersectionObserver`), e `LazyImage.jsx` (`imgRef` per accedere al DOM).

React Router (react-router-dom v6)

Libreria per il routing client-side. Ogni URL corrisponde a un componente React.

```
import { BrowserRouter, Routes, Route, Link, useParams, useSearchParams } from 'react-router-dom'

// Configurazione route
<BrowserRouter>
  <Routes>
    <Route path="/" element={<HomePage />} />
```

```

    <Route path="/corpi-celesti" element={<CorpiLista />} />
    <Route path="/corpi-celesti/:slug" element={<CorpoDettaglio />} />
    <Route path="*" element={<NotFound />} />
  </Routes>
</BrowserRouter>

// Navigazione
<Link to="/corpi-celesti">Vai alla lista</Link>;

// Hook per leggere i parametri
function CorpoDettaglio() {
  const { slug } = useParams();           // /corpi-celesti/:slug
  const [searchParams] = useSearchParams(); // ?page=1&cat=terra
  const location = useLocation();          // URL corrente
}

```

Componenti chiave: `BrowserRouter` (wrappa l'app), `Routes` (contenitore route), `Route` (singola route), `Link` (navigazione). Hook: `useParams` (parametri URL), `useSearchParams` (query string), `useLocation` (URL corrente). In Astralis: `App.jsx` definisce 5 route. `Comparatore.jsx` usa `useSearchParams` per lo state via URL. `CorpoDettaglio.jsx` usa `useParams` per lo slug.

Controlled vs Uncontrolled Components

Controlled: il valore dell'input è controllato da React (`useState`). **Uncontrolled:** il valore è gestito dal DOM.

```

// Controlled — valore nello state di React
function SearchBar() {
  const [query, setQuery] = useState("");
  return <input value={query} onChange={(e) => setQuery(e.target.value)} />;
}
// Vantaggio: puoi validare, formattare, submit programmaticamente

// Uncontrolled — valore nel DOM
function SearchBar() {
  const inputRef = useRef(null);
  const getQuery = () => inputRef.current.value;
  return <input ref={inputRef} />;
}
// Vantaggio: meno codice, utile per form semplici

```

In React: il 90% dei casi usa **controlled** (più prevedibile). In Astralis: `SearchBar.jsx` , `Comparatore.jsx` , tutti i form admin sono controlled. Uncontrolled si usa solo con `useRef` per accedere al DOM direttamente.

createRoot (React 18)

Entry point moderno di React 18. Sostituisce il vecchio `ReactDOM.render()` .

```
// React 17 (obsoleto)
import ReactDOM from "react-dom";
ReactDOM.render(<App />, document.getElementById("root"));

// React 18 (moderno)
import { createRoot } from "react-dom/client";
const root = createRoot(document.getElementById("root"));
root.render(<App />);
```

`createRoot()` crea il root del rendering. Supporta le Concurrent Features di React 18 (StrictMode, Suspense, transitions). In Astralis: `main.jsx` usa `createRoot(document.getElementById('root')).render(...)` con `<React.StrictMode>` .

Domande React — Durante la Demo del Progetto

Queste domande vengono fatte **mentre mostri il progetto**. Preparati a rispondere rapidamente.

Q: Perché hai usato React per il guest e Blade per l'admin?

Il guest è una SPA standalone che beneficia di transizioni fluide, state management lato client, e reattività senza ricaricare la pagina. L'admin è un CRUD classico dove Blade + Alpine.js è più semplice, performante, e non richiede un framework pesante. Inertia era stato installato inizialmente ma rimosso perché creava un livello intermedio inutile.

Q: Come funziona il routing lato client?

`react-router-dom` definisce 5 route in `App.jsx` : `/` (home), `/corpi-celesti` (lista), `/corpi-celesti/:slug` (dettaglio), `/confronta` (comparatore), `/*` (404). Laravel cattura tutto con `Route::get('/{any}')` e passa il controllo a React.

Q: Cosa fa `apiClient.js` ?

Un'istanza di axios con configurazione centralizzata: base URL `/api` , timeout 30s, retry automatico (2 tentativi), interceptors per gestire errori. Il Vite proxy inoltra le chiamate `/api` a `localhost:8000` .

Retry logic in `apiClient.js` — come funziona il backoff esponenziale:

1. **Quando retrya:** solo su errori di rete (`!error.response`) o server error (`status >= 500 && < 600`). I 4xx (403, 404, 422) NON vengono retryati — sono errori client, riprovare

non serve.

2. **Backoff esponenziale:** $delay = 2^n \times 500ms$ (n = tentativo). Primo retry: 1s. Secondo retry: 2s. Evita flooding del server.
3. **Abort check:** prima di ogni retry verifica `config.signal?.aborted`. Se il componente si è smontato (AbortController abortito), non riprova — evita “setState on unmounted component”.
4. **Max 2 retry:** dopo 2 tentativi falliti, l'errore viene passato al componente che mostra fallback UI.
5. **Skip su cancel:** se l'errore è un `Cancel` (AbortController), lo rilancia subito senza retry.

```
// Codice chiave dell'interceptor
const shouldRetry =
  !error.response ||
  (error.response.status >= 500 && error.response.status < 600);
if (shouldRetry && retryCount <= 2) {
  const delay = Math.pow(2, retryCount) * 500; // 1s, 2s
  await new Promise((resolve) => setTimeout(resolve, delay));
  return apiClient.request(retryConfig);
}
```

Q: Come funziona `useFetch` ?

Custom hook con `useReducer` per gestire loading, error, data. Usa `AbortController` nel `useEffect` per cancellare fetch in corso quando il componente si smonta (evita memory leak e “setState on unmounted component”). Il pattern `START` preserva i dati esistenti durante ricaricamenti.

Q: Perché `ErrorBoundary` è una class component?

Perché solo le class components possono usare `componentDidCatch()` e `getDerivedStateFromError()`. Le function components non hanno lifecycle methods per catturare errori nei figli. In React 19+ ci sono le error boundaries tramite hook, ma in React 18 servono le class.

Q: Come animi il sistema solare?

`requestAnimationFrame` + direct DOM manipulation via refs. Un angolo cresce infinitamente per ogni pianeta, convertito in coordinate x/y con `Math.sin()` e `Math.cos()`. Ogni pianeta ha velocità e raggio differenziati. Zero re-render durante l'animazione perché manipoliamo il DOM direttamente, non lo state React. Più performante di `setInterval` perché si sincronizza con il refresh rate del display.

Q: Perché usi `React.memo()` su `LightboxGalleria`?

Perché quando il Lightbox è chiuso e riaperto, se le props non cambiano, il componente non deve re-renderizzare. Il re-render di `LightboxGalleria` è costoso (molti figli, immagini grandi). `React.memo()` confronta le props e salta il re-render se invariate.

Q: Cosa succede se la API fallisce?

`useFetch` cattura l'errore e lo stato `error` viene popolato. Il componente mostra un messaggio di errore con opzione "Riprova" che ricarica i dati. Se il componente crasha, `ErrorBoundary` in `App.jsx` mostra una fallback UI con pulsante retry che resetta lo stato.

Q: Come comunica React con Laravel?

React chiama `/api/corpi-celesti` Vite proxy (in `vite.config.js`) inoltra a `localhost:8000` controller API in Laravel esegue la query restituisce JSON React riceve e renderizza. Nessun Inertia (rimosso), comunicazione pura via API REST.

10. CORS & Sicurezza

CORS (Cross-Origin Resource Sharing)

Il browser blocca richieste da un dominio a un altro. In Astralis: React (`localhost:5175`) chiama Laravel (`localhost:8000`).

Soluzione: Vite proxy in `vite.config.js` :

```
server: {  
  proxy: { '/api': 'http://localhost:8000' }  
}
```

In produzione: Laravel 13 gestisce automaticamente con middleware `HandleCors` .

CSRF (Cross-Site Request Forgery)

Laravel genera un token CSRF per ogni sessione. Le form Blade lo includono con `@csrf` . Le API REST non lo usano (stateless).

Throttle

Rate limiting: `throttle:120,1` = max 120 richieste/minuto sulle route admin. `throttle:60,1` sulle API. `throttle:30,1` su `suggestNome`.

Auth

Laravel Breeze gestisce sessioni. Middleware `auth` protegge le route. `verified` richiede email verificata.

Authorization

Policy + Gate. Admin bypassa tutto con `before()` returning `true`. Non-admin: `create/update/delete` restituiscono `false`.

11. Errori Comuni da NON Fare

Errore	Perché è sbagliato	Cosa fare invece
<code>Image::read()</code> in Intervention v4	La facade è stata rimossa in v4	<code>new ImageManager(new Driver())->decodePath(\$path)</code>
<code>resize()</code> invece di <code>scaleDown()</code>	<code>resize()</code> forza le dimensioni e distorce	<code>scaleDown(width, height)</code> preserva aspect ratio
Dimenticare <code>Http::fake()</code> nei test	L'observer dispatcha job NASA test falliti	<code>Http::fake()</code> in <code>setUp()</code>
Dimenticare eager loading nelle show	N+1 problem: N query extra	<code>::with(['relazione'])->firstOrFail()</code>
Usare <code>->get()</code> invece di <code>->paginate()</code>	Carica tutto in memoria	<code>->paginate(20)</code> con <code>->withQueryString()</code>
<code>@if</code> senza <code>@endif</code> in Blade	Errore 500 silenzioso	Controllare chiusura if/foreach
Dimenticare <code>->constrained()</code> FK	FK non reale	<code>\$table->foreignId('x_id')->constrained()</code>
Mantenere Inertia quando non serve	Dipendenze inutili, conflitti routing	Rimuovere se SPA standalone
<code>\$fillable</code> senza tutti i campi	Mass assignment exception	Aggiungere TUTTI i campi modificabili

12. Live Coding — 12 Esercizi con Soluzione

Esercizio 1: Route + Controller + View

Traccia: Crea una route `/test` che mostra una pagina con un messaggio.

Soluzione:

```
// routes/web.php (aggiungere prima del catch-all)
Route::get('/test', function () {
```

```
return view('test');
}->name('test');
```

```
// resources/views/test.blade.php
@extends('admin.layouts.app')
@section('title', 'Test')
@section('content')
    <h1>Ciao dal progetto Astralis!</h1>
    <p>Questo è un endpoint di test.</p>
@endsection
```

Variante controller:

```
// app/Http/Controllers/TestController.php
<?php
namespace App\Http\Controllers;

use Illuminate\View\View;

class TestController extends Controller
{
    public function __invoke(): View
    {
        $data = ['nome' => 'Astralis', 'versione' => '13'];
        return view('test', compact('data'));
    }
}
```

```
// routes/web.php
use App\Http\Controllers\TestController;
Route::get('/test', TestController::class)->name('test');
```

Esercizio 2: Array filter/map

Traccia: Data un array di oggetti, filtra quelli con prezzo > 10 e mappa in array di nomi.

Soluzione:

```
$prodotti = [
    ['nome' => 'Penna', 'prezzo' => 5],
    ['nome' => 'Quaderno', 'prezzo' => 12],
```



```
['nome' => 'Matita', 'prezzo' => 3],
['nome' => 'Libro', 'prezzo' => 25],
];

// Filter solo prezzo > 10
$filtrati = array_filter($prodotti, fn($p) => $p['prezzo'] > 10);

// Map solo nomi
$nomi = array_map(fn($p) => $p['nome'], $filtrati);

// Risultato: ['Quaderno', 'Libro']
```

Variante con collect (Laravel):

```
$nomi = collect($prodotti)
->filter(fn($p) => $p['prezzo'] > 10)
->pluck('nome')
->toArray();
```

Esercizio 3: Somma array di oggetti

Traccia: Calcola la somma totale di un array di oggetti con campo `valore` .

Soluzione:

```
$transazioni = [
    ['descrizione' => 'Acquisto', 'valore' => 100],
    ['descrizione' => 'Vendita', 'valore' => 250],
    ['descrizione' => 'Spesa', 'valore' => -50],
];

// PHP base
$somma = 0;
foreach ($transazioni as $t) {
    $somma += $t['valore'];
}
// Risultato: 300

// Con array_sum + array_column
$somma = array_sum(array_column($transazioni, 'valore'));
```

```
// Con collect (Laravel)
$somma = collect($transazioni)->sum('valore');
```

Esercizio 4: Somma + Filter + Map (combinato)

Traccia: L'esaminatore ti fornisce un array di oggetti. Devi:

1. Calcolare la somma totale di un campo numerico
2. Filtrare gli elementi che superano una soglia
3. Creare un nuovo array con solo i campi richiesti

Dati dell'esaminatore:

```
$prodotti = [
    ['nome' => 'Penna', 'prezzo' => 5, 'categoria' => 'cancelleria'],
    ['nome' => 'Quaderno', 'prezzo' => 12, 'categoria' => 'cancelleria'],
    ['nome' => 'Matita', 'prezzo' => 3, 'categoria' => 'cancelleria'],
    ['nome' => 'Libro', 'prezzo' => 25, 'categoria' => 'libri'],
    ['nome' => 'Zaino', 'prezzo' => 40, 'categoria' => 'accessori'],
    ['nome' => 'Astuccio', 'prezzo' => 8, 'categoria' => 'accessori'],
];
```

Step 1 — Somma totale:

```
$sommaTotale = collect($prodotti)->sum('prezzo'); // 93
```

Step 2 — Filtra per prezzo > 10:

```
$filtrati = collect($prodotti)
    ->filter(fn($p) => $p['prezzo'] > 10)
    ->values()
    ->toArray();
```

Step 3 — Mappa: solo nomi dei filtrati:

```
$nomi = array_map(fn($p) => $p['nome'], $filtrati);
// ['Quaderno', 'Libro', 'Zaino']
```

Step 4 — Somma solo dei filtrati:

```
$sommaFiltrati = collect($filtrati)->sum('prezzo'); // 77
```

Soluzione PHP base (senza collect):

```
$sommaTotale = 0;
$filtrati = [];
foreach ($prodotti as $p) {
    $sommaTotale += $p['prezzo'];
    if ($p['prezzo'] > 10) {
        $filtrati[] = $p;
    }
}
$nomi = array_map(fn($p) => $p['nome'], $filtrati);
```

Variante esame: “Calcola la media dei prezzi, poi restituisci solo gli articoli sopra la media.”

```
$media = collect($prodotti)->avg('prezzo'); // 15.5
$sopraMedia = collect($prodotti)
    ->filter(fn($p) => $p['prezzo'] > $media)
    ->values()
    ->toArray();
// [{nome: 'Libro', prezzo: 25}, {nome: 'Zaino', prezzo: 40}]
```

Esercizio 5: Route POST + Form + Validazione

Traccia: Crea un form che accetta un nome e un'email, li valida, e li salva in sessione. Mostra un messaggio di successo.

Soluzione:

```
// routes/web.php — PRIMA del catch-all
Route::get('/contatti', function () {
    return view('contatti');
})->name('contatti.form');

Route::post('/contatti', function (\Illuminate\Http\Request $request) {
    $validated = $request->validate([
        'nome' => 'required|string|max:255',
        'email' => 'required|email|max:255',
    ]);

    $contatti = session()->get('contatti', []);
    $contatti[] = $validated;
    session()->put('contatti', $contatti);
});
```

```
return redirect()->route('contatti.form')
    ->with('success', 'Contatto salvato con successo!');
}->name('contatti.store');
```

```
{{-- resources/views/contatti.blade.php --}}
@extends('admin.layouts.app')
@section('title', 'Contatti')
@section('content')
    <h1>Aggiungi Contatto</h1>

    @if (session('success'))
        <div class="bg-green-500/20 text-green-300 p-3 rounded">
            {{ session('success') }}
        </div>
    @endif

    <form method="POST" action="{{ route('contatti.store') }}">
        @csrf
        <input name="nome" value="{{ old('nome') }}" placeholder="Nome" class="admin-input">
        @error('nome') <span class="text-red-400">{{ $message }}</span> @enderror

        <input name="email" value="{{ old('email') }}" placeholder="Email" class="admin-input">
        @error('email') <span class="text-red-400">{{ $message }}</span> @enderror

        <button type="submit" class="bg-admin-primary text-admin-bg px-4 py-2 rounded">
            Salva
        </button>
    </form>
@endsection
```

Pattern chiave: @csrf (token sicurezza), \$request->validate() (validazione), old('nome') (preserva input), @error (mostra errori), session()->put() (salva in sessione), ->with('success') (flash message).

Esercizio 6: array_reduce

Traccia: Data un array di transazioni, calcola il totale usando array_reduce (invece di array_sum + array_column).

Soluzione:

```

$transazioni = [
    ['descrizione' => 'Stipendio', 'valore' => 2500],
    ['descrizione' => 'Affitto', 'valore' => -800],
    ['descrizione' => 'Spesa', 'valore' => -150],
    ['descrizione' => 'Vendita', 'valore' => 300],
];

// array_reduce — il callback riceve $carry (accumulatore) e $item
$somma = array_reduce($transazioni, function ($carry, $item) {
    return $carry + $item['valore'];
}, 0); // 0 = valore iniziale di $carry

// Risultato: 1850 (2500 - 800 - 150 + 300)

// Arrow function (più concisa)
$somma = array_reduce($transazioni, fn($carry, $item) => $carry + $item['valore'], 0);

```

Cos'è `array_reduce` : riduce un array a un singolo valore. Il callback riceve due parametri: `$carry` (il valore accumulato finora) e `$item` (l'elemento corrente). Restituisce il nuovo `$carry` dopo ogni iterazione.

Esempio avanzato — Raggruppa per tipo:

```

// Raggruppa transazioni per segno (positive/negative)
$raggruppate = array_reduce($transazioni, function ($carry, $item) {
    $chiave = $item['valore'] >= 0 ? 'positive' : 'negative';
    $carry[$chiave][] = $item;
    return $carry;
}, []);

// Risultato:
// [
//   'positive' => [['descrizione' => 'Stipendio', 'valore' => 2500], ['descrizione' => 'Vendita', 'valore' => 300]],
//   'negative' => [['descrizione' => 'Affitto', 'valore' => -800], ['descrizione' => 'Spesa', 'valore' => -150]]
// ]

```

Variante con Collection Laravel:

```

$somma = collect($transazioni)->sum('valore');
$raggruppate = collect($transazioni)->groupBy(fn($t) => $t['valore'] >= 0 ? 'positive' : 'negative');

```

Esercizio 7: Query Eloquent (Ricerca nel Database)

Traccia: Trova tutti i corpi celesti di una specifica categoria, ordinati per nome. Poi conta quanti ci sono.

Soluzione:

```
use App\Models\CorpoCeleste;

// Query base: filtra per categoria_id = 1 (Pianeta)
$corpi = CorpoCeleste::where('categoria_id', 1)
    ->orderBy('nome')
    ->get();
// SELECT * FROM corpi_celesti WHERE categoria_id = 1 ORDER BY nome

echo $corpi->count(); // 8 (Terra, Marte, Giove, ...)
echo $corpi->first()->nome; // "Giove" (ordine alfabetico)

// Con eager loading (evita N+1)
$corpi = CorpoCeleste::with('categoria') // Carica anche la categoria
    ->where('categoria_id', 1)
    ->orderBy('nome')
    ->get();

// Filtra + conta in una riga
$total = CorpoCeleste::where('categoria_id', 1)->count();

// Usa lo slug invece dell'ID (Route Model Binding)
$corpi = CorpoCeleste::whereHas('categoria', fn($q) => $q->where('slug', 'pianeta'))
    ->orderBy('nome')
    ->get();
```

Pattern comuni Eloquent:

Operazione	Codice	Risultato
Filtra	<code>->where('campo', \$val)</code>	WHERE campo = val
Ordina	<code>->orderBy('nome')</code>	ORDER BY nome ASC
Primo	<code>->first()</code>	Primo record
Conta	<code>->count()</code>	Numero record
Esiste	<code>->exists()</code>	bool (più veloce di count)

Trova per ID	->find(\$id)	Record o null
Trova o errore	->findOrFail(\$id)	Record o 404

In Astralis: l'API `/api/corpi-celesti?categoria=pianeta` usa esattamente questo pattern: filtra per slug categoria, ordina per nome, pagina con `paginate(12)`.

Esercizio 8: Migration — Creare una Tabella

Traccia: Crea una migration per una tabella `premi` con: id, nome (string), corpo_celeste_id (foreign key con `restrictOnDelete`), anno (year), descrizione (text nullable), timestamps.

Soluzione:

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('premi', function (Blueprint $table) {
            $table->id();
            $table->string('nome');
            $table->foreignId('corpo_celeste_id')
                ->constrained('corpi_celesti')
                ->restrictOnDelete(); // Non puoi cancellare il corpo se ha premi
            $table->year('anno');
            $table->text('descrizione')->nullable();
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('premi');
    }
};
```

Note chiave:

- `foreignId('corpo_celeste_id')->constrained()` crea la FK automaticamente (colonna `unsignedBigInteger` + vincolo)

- `restrictOnDelete()` = blocca cancellazione del padre se figli esistono
- `cascadeOnDelete()` = elimina i figli quando cancelli il padre
- `$table->nullable()` = il campo può essere null
- `down()` deve essere l'opposto di `up()` — `dropIfExists` rimuove la tabella

Cambio tabella esistente (`Schema::table`):

```
Schema::table('premi', function (Blueprint $table) {
    $table->string('premio_internazionale')->nullable()->after('nome'); // Aggiunge colonna
    $table->dropColumn('descrizione'); // Rimuove colonna
});
```

Esercizio 9: Policy — Autorizzazione

Traccia: Scrivi una Policy per l'entità `Missione` con: `bypass admin` nel `before()` , permessi per `viewAny` , `create` , `update` , `delete` .

Soluzione:

```
<?php
declare(strict_types=1);

namespace App\Policies;

use App\Models\Missione;
use App\Models\User;
use Illuminate\Auth\Access\HandlesAuthorization;

class MissionePolicy
{
    use HandlesAuthorization;

    // Admin bypassa TUTTI i controlli
    public function before(User $user): ?bool
    {
        if ($user->is_admin) {
            return true; // Consentito sempre
        }
        return null; // Passa al metodo specifico
    }

    // Chiunque (autenticato) può vedere la lista
    public function viewAny(User $user): bool
    {

```



```

        return true;
    }

    // Solo admin può creare
    public function create(User $user): bool
    {
        return $user->is_admin;
    }

    // Solo admin può modificare
    public function update(User $user, Missione $missione): bool
    {
        return $user->is_admin;
    }

    // Solo admin può eliminare
    public function delete(User $user, Missione $missione): bool
    {
        return $user->is_admin;
    }
}

```

Registrazione in `AuthServiceProvider` :

```

protected $policies = [
    Missione::class => MissionePolicy::class,
];

```

Uso nel controller:

```

$this->authorize('update', $missione); // Lancia 403 se non autorizzato

```

Esercizio 10: Observer — Eventi Eloquent

Traccia: Scrivi un Observer per `Missione` che, quando una missione viene creata, logga un messaggio e dispatcha un job. Disabilita il comportamento in testing.

Soluzione:

```

<?php
declare(strict_types=1);

namespace App\Observers;

```

```

use App\Models\Missione;
use App\Jobs\ImportMissionImages;
use Illuminate\Support\Facades\Log;

class MissioneObserver
{
    public function created(Missione $missione): void
    {
        // Skip in testing — evita side effects nei test
        if (app()->environment('testing')) {
            return;
        }

        Log::info("Missione creata: {$missione->nome}");

        // Dispatch job per importare immagini
        ImportMissionImages::dispatch($missione);
    }

    public function updated(Missione $missione): void
    {
        if (app()->environment('testing')) {
            return;
        }

        Log::info("Missione aggiornata: {$missione->nome}");
    }

    public function deleted(Missione $missione): void
    {
        if (app()->environment('testing')) {
            return;
        }

        Log::info("Missione eliminata: {$missione->nome}");
    }
}

```

Registrazione in `AppServiceProvider::boot()` :

```

use App\Models\Missione;
use App\Observers\MissioneObserver;

```

```
public function boot(): void
{
    Missione::observe(MissioneObserver::class);
}
```

Pattern chiave: `app()->environment('testing')` impedisce che l'Observer esegua codice reale durante i test. Senza questo guard, ogni `factory()->create(Missione::class)` dispatcherebbe un job HTTP reale.

Esercizio 11: Controller store() — Validazione + Salvataggio

Traccia: Scrivi il metodo `store()` di un controller che valida i dati, gestisce l'upload di un'immagine, crea il record, e reindirizza con messaggio di successo.

Soluzione:

```
use App\Models\Missione;
use App\Services\ImageUploadService;
use Illuminate\Http\Request;

public function store(Request $request): RedirectResponse
{
    // 1. Validazione
    $validated = $request->validate([
        'nome' => 'required|string|max:255',
        'agenzia' => 'required|string|max:255',
        'data_lancio' => 'required|date',
        'stato' => 'required|in:Completata,In corso,Pianificata',
        'logo' => 'nullable|image|mimes:jpeg,png,jpg|max:300', // max 300KB
        'sito_web' => 'nullable|url',
    ]);

    // 2. Upload immagine (se fornita)
    if ($request->hasFile('logo')) {
        $validated['logo'] = ImageUploadService::upload(
            $request->file('logo'),
            300 // Max dimensione in KB
        );
    }

    // 3. Creazione record
    $missione = Missione::create($validated);
```

```
// 4. Redirect con messaggio
return redirect()
->route('admin.missioni.show', $missione)
->with('success', "Missione '{$missione->nome}' creata con successo.");
}
```

Pattern chiave:

- `$request->validate()` lancia `ValidationException` automaticamente (redirect back con errori)
- `hasFile()` verifica che il file esista prima di processarlo
- `ImageUploadService::upload()` gestisce validazione, ridimensionamento, e salvataggio
- `with('success', ...)` imposta un flash message disponibile nella view con `@session('success')`
- Il metodo restituisce `RedirectResponse` (non `view`)

Esercizio 12: Route + View + Array (ciclo for + media)

Pattern d'esame reale: l'esaminatore chiede di creare una nuova route e view, poi fare PHP live coding inside (ciclo for su array + calcolo media). Il collega ha avuto esattamente questa traccia.

Traccia: Crea una route `/esercizio` che passa un array di voti a una view. Nella view, usa un ciclo `for` per calcolare la media e stampa il risultato.

Soluzione — Step 1: Route (`routes/web.php` , PRIMA del catch-all):

```
Route::get('/esercizio', function () {
    $voti = [7, 8, 6, 9, 10, 5, 8];
    return view('esercizio', compact('voti'));
})->name('esercizio');
```

Soluzione — Step 2: View (`resources/views/esercizio.blade.php`):

```
@extends('admin.layouts.app')
@section('title', 'Esercizio Voti')
@section('content')
    <h1 class="text-2xl font-bold mb-4">Calcolo Media Voti</h1>

    @php
        $somma = 0;
        $n = count($voti);
        for ($i = 0; $i < $n; $i++) {
            $somma += $voti[$i];
        }
    @endphp
```

```

    }
    $media = $somma / $n;
@endphp

<p>Voti: {{ implode(' ', $voti) }}</p>
<p>Somma: {{ $somma }}</p>
<p>N. voti: {{ $n }}</p>
<p class="text-xl font-bold">Media: {{ $media }}</p>
@endsection

```

Output: Voti: 7, 8, 6, 9, 10, 5, 8 Somma: 53 N. voti: 7 Media: 7.571428...

Varianti (le più chieste dall'esaminatore):

Variante A — Voto massimo e minimo:

```

@php
    $max = $voti[0];
    $min = $voti[0];
    for ($i = 1; $i < count($voti); $i++) {
        if ($voti[$i] > $max) $max = $voti[$i];
        if ($voti[$i] < $min) $min = $voti[$i];
    }
@endphp
<p>Massimo: {{ $max }} — Minimo: {{ $min }}</p>

```

Variante B — Contare quanti superano una soglia:

```

@php
    $soglia = 6;
    $conta = 0;
    for ($i = 0; $i < count($voti); $i++) {
        if ($voti[$i] >= $soglia) {
            $conta++;
        }
    }
@endphp
<p>Voti >= {{ $soglia }}: {{ $conta }} su {{ count($voti) }}</p>

```

Variante C — Ordinare l'array:

```

@php
    $ordinati = $voti;

```

```
sort($ordinati); // Ascendente: sort() — Discendente: rsort()
@endphp
<p>Ordinati: {{ implode(' ', $ordinati) }}</p>
```

Variante D — Somma + media senza for (Laravel/PHP conciso):

```
@php
    $somma = array_sum($voti);
    $n = count($voti);
    $media = $somma / $n;
@endphp
```

Variante E — Array associativo (oggetti):

```
@php
    $studenti = [
        ['nome' => 'Marco', 'voto' => 7],
        ['nome' => 'Lucia', 'voto' => 9],
        ['nome' => 'Paolo', 'voto' => 6],
    ];
    $somma = 0;
    for ($i = 0; $i < count($studenti); $i++) {
        $somma += $studenti[$i]['voto'];
    }
    $media = $somma / count($studenti);
@endphp

<ul>
    @for ($i = 0; $i < count($studenti); $i++)
        <li>{{ $studenti[$i]['nome'] }}: {{ $studenti[$i]['voto'] }}</li>
    @endfor
</ul>

<p>Media: {{ $media }}</p>
```

Consigli live coding:

1. Parti dalla route — è sempre il primo passo (`Route::get('/nome', function() { ... })`)
 2. Passa i dati con `compact('var')` o `['var' => $valore]`
 3. Nella view usa `@php ... @endphp` per la logica PHP
 4. Usa `@for / @endfor` se l'esaminatore vuole il ciclo nella view, oppure `@php for(...) { } @endphp`
 5. Se ti blocca: parti dalla soluzione PHP base (`for` + variabili), poi mostra la versione Laravel (`collect()->avg()`)
-

13. Comandi Rapidi

```
# Avvio
php artisan serve          # Backend   localhost:8000
npm run dev                # Frontend  localhost:5175

# Database
php artisan migrate:fresh --seed # Reset completo

# Test
php artisan test           # 271 PHPUnit
npm test                   # 110 Vitest

# NASA
php artisan astralis:fetch-nasa # Import immagini
php artisan astralis:gallery --fix # Ripara galleria

# Credenziali
# admin@astralis.it / password
```

14. Credenziali

Ruolo	Email	Password
Admin	admin@astralis.it	password
Utente normale	(registrazione libera su /register)	—

15. Mappa Navigazione VS Code — “Dove Trovi Cosa”

Regola d'oro: quando l'esaminatore chiede “dove hai messo X?”, apri direttamente il file corretto. Non cercare — sai dove è tutto.

Architettura in 30 Secondi

```
astralis/
├── app/          # Backend PHP (Laravel)
│   ├── Http/Controllers/ # Logica HTTP
│   ├── Models/         # Tabelle DB
│   └── Services/       # Logica business
```

```

Policies/      # Autorizzazione
Observers/     # Eventi Eloquent
Jobs/          # Task in background
Resources/     # API JSON transformation
database/
  migrations/  # Struttura tabelle
  factories/   # Dati fake test
  seeders/    # Dati reali demo
routes/        # Definizione route
  web.php     # Admin + catch-all SPA
  api.php     # API JSON pubbliche
  auth.php    # Login/Register Breeze
resources/
  views/admin/ # Blade admin
  js/guest/    # React SPA
tests/         # 271 test PHP

```

Tabella Rapida — “Dove Cerchi?”

Domanda dell'esaminatore	File da aprire	Cosa trovare
“Dove sono le route API?”	routes/api.php	10 endpoint GET, tutti pubblici
“Dove sono le CRUD admin?”	routes/web.php	Route::resource(...) nel gruppo admin
“Dove sono le route auth?”	routes/auth.php	Login, register, profilo — Breeze
“Dove i controller delle CRUD?”	app/Http/Controllers/Admin/	8 file: CorpoCelesteController, CategoriaController, ecc.
“Dove i controller API?”	app/Http/Controllers/Api/	6 file: CorpoCelesteController, ecc.
“Dove il model CorpoCeleste?”	app/Models/CorpoCeleste.php	Fillable, casts, relazioni (belongsTo, hasMany, belongsToMany)

“Dove le migrations?”	database/migrations/	21 file con timestamp. Cerca create_corpi_celesti_table
“Dove le factory?”	database/factories/	5 file: CorpoCelesteFactory, CategoriaFactory, ecc.
“Dove i seeder?”	database/seeder/	DatabaseSeeder + 7 seeder specifici
“Dove i Service?”	app/Services/	NasalImageService, WordMapService, ImageUploadService
“Dove le Policy?”	app/Policies/	5 file: CorpoCelestePolicy, CategoriaPolicy, ecc.
“Dove l’Observer?”	app/Observers/CorpoCelesteObserver.php	created() dispatcha job NASA
“Dove il Job NASA?”	app/Jobs/ImportNasalImage.php	ShouldBeUnique, handle(), failed()
“Dove le FormRequest?”	app/Http/Requests/	13 file: Store/Update per ogni entità
“Dove le API Resource?”	app/Http/Resources/	5 file: CorpoCelesteResource, ecc.
“Dove il layout admin?”	resources/views/admin/layouts/app.blade.php	Master layout con sidebar + Alpine.js
“Dove il form CorpoCeleste?”	resources/views/admin/corpi-celesti/_form.blade.php	6 sezioni, 18 campi
“Dove la sidebar?”	resources/views/admin/partials/_sidebar-nav.blade.php	Lettura da config/admin.php
“Dove la config admin?”	config/admin.php	nav_items, color_presets, mission_stati
“Dove il gate admin?”	app/Providers/AuthServiceProvider.php	Gate::define('admin', ...)
“Dove la dashboard?”	resources/views/admin/dashboard.blade.php	4 stat card + 3 grafici Chart.js

"Dove l'entry React?"	<code>resources/js/guest/main.jsx</code>	ReactDOM.createRoot + BrowserRouter
"Dove il routing React?"	<code>resources/js/guest/App.jsx</code>	5 route: /, /corpi-celesti, /:slug, /confronta, /*
"Dove la homepage React?"	<code>resources/js/guest/pages/HomePage.jsx</code>	Hero + SolarSystem + In Evidenza
"Dove la lista corpi?"	<code>resources/js/guest/pages/CorpiLista.jsx</code>	Griglia card, filtri, paginazione
"Dove il dettaglio?"	<code>resources/js/guest/pages/CorpoDettaglio.jsx</code>	Metriche, lightbox, curiosità, timeline, simili
"Dove il comparatore?"	<code>resources/js/guest/pages/Comparatore.jsx</code>	Tabella confronto 2 corpi su 7 metriche
"Dove il sistema solare?"	<code>resources/js/guest/components/SolarSystem.jsx</code>	8 pianeti con requestAnimationFrame
"Dove i componenti React?"	<code>resources/js/guest/components/</code>	CorpoCard, LightboxGalleria, TimelineMissioni, ecc.
"Dove gli hook React?"	<code>resources/js/guest/hooks/</code>	useFetch (useReducer + AbortController), useDebounce
"Dove apiClient?"	<code>resources/js/guest/apiClient.js</code>	axios + retry + timeout 30s
"Dove la config Vite?"	<code>vite.config.js</code>	React plugin + proxy /api localhost:8000
"Dove i test PHP?"	<code>tests/</code>	271 test: Feature/, Unit/, TestCase base
"Dove i test React?"	<code>resources/js/guest/</code> + <code>*.test.jsx</code>	110 test Vitest: componenti + integrazione
"Dove i CSS?"	<code>resources/css/app.css</code>	CSS variables admin, Tailwind, x-cloak
"Dove il CSS del guest?"	In Tailwind via classi inline + CSS in app.css	Nessun file separato

Live coding: route + view + array	<pre> routes/web.php Route::get('/esercizio', fn() => view('esercizio', compact('voti'))) + resources/views/esercizio.blade.php con @for / @php for() </pre>	Media, max, min, conta soglia
--	--	-------------------------------

Comandi Rapidi per VS Code

```

Ctrl+P  "nome_file"    # Apri file rapidamente
Ctrl+Shift+F  "testo"    # Cerca in tutto il progetto
Ctrl+Shift+R  "classe"   # Cerca classi/interfacce
F12          # Vai alla definizione
Alt+Left     # Torna indietro
Ctrl+`       # Toggle terminale

```

Consigli per la Demo

1. **Parti sempre dalla route:** quando l'esaminatore chiede "dove X?", prima mostra la route, poi il controller, poi la view
2. **Usa Ctrl+P** per navigare rapidamente — non usare l'alberatura laterale
3. **Mostra il file reale**, non solo spiegare a parole
4. **Se non trovi il file:** usa Ctrl+Shift+F per cercare il nome della classe/fun
5. **Tieni aperto:** routes/web.php , routes/api.php , app/Http/Controllers/Admin/ — sono i più richiesti

16. Mini Simulazione Esame — 15 Minuti

Esercitati con un collega che fa le domande. Rispondi ad alta voce, come se fosse l'esaminatore.

Sequenza domande (simula il ritmo reale)

#	Domanda	Risposta attesa	Tempo
1	"Spiega il progetto"	Script apertura: login dashboard CRUD React guest	2 min
2	"Dove sono le route API?"	Apri routes/api.php , mostra i 10 endpoint	30 sec
3	"Dove i controller CRUD?"	Apri app/Http/Controllers/Admin/ , mostra struttura	30 sec

4	“Cos’è una migration?”	Definizione + mostra file in <code>database/migrations/</code>	1 min
5	“Cos’è una classe in PHP?”	Definizione + esempio <code>CorpoCeleste extends Model</code>	1 min
6	“Differenza == e ===?”	Loose vs strict, consiglio: usa sempre ===	30 sec
7	Live coding: crea route + view, calcola media array con for	<code>Route::get('/esercizio', fn() => view('esercizio', compact('voti')))</code> + ciclo <code>@for</code> nella view	5 min
8	Live coding: trova il voto massimo con un ciclo for	Inizializza <code>\$max = \$arr[0]</code> , confronta nel ciclo, stampa il risultato	3 min
9	“Perché React per il guest?”	SPA, transizioni fluide, reattività senza reload	1 min
10	“Come funziona il routing React?”	react-router-dom + catch-all in Laravel	1 min
11	“Cos’è un Observer?”	<code>CorpoCelesteObserver</code> dispatcha job NASA su <code>created</code>	1 min
12	“Cos’è una Foreign Key?”	Colonna che collega tabelle, <code>constrained()</code> , integrità referenziale	30 sec
13	“Come testi il progetto?”	PHPUnit + <code>Http::fake()</code> + Factory, Vitest per React	1 min

Tempo totale: ~15 minuti. L’esame reale dura ~1 ora, quindi ci sono molte altre domande. Questa simulazione copre i punti chiave.

Consigli per la Performance

1. **Non dire “non lo so”** — se non sai esattamente, spiega il concetto generale e cerca il file corretto
2. **Mostra il codice, non solo spiegare** — apri il file, mostra la riga esatta
3. **Collega sempre alla teoria** — quando mostri una route, spiega il pattern REST
4. **Usa il tuo progetto come esempio** — non usare esempi generici, usa Astralis
5. **Se ti blocca un live coding:** parti dalla soluzione semplice (PHP base), poi ottimizza con Laravel/Collection
6. **Gestisci il tempo:** non restare troppo su una domanda — se non sai, vai avanti e tornaci dopo

Errori Comuni da Evitare

Errore	Perché è grave	Cosa dire invece
Confondere route web e API	Mostra che non capisci il dual rendering	"web.php restituisce HTML, api.php restituisce JSON"
Dire "non usiamo le migrations"	Le migrations sono fondamentali in Laravel	Mostra il file e spiega up()/down()
Dimenticare <code>Http::fake()</code> nei test	L'observer fa chiamate HTTP reali	Spiega il testing guard nell'observer
Confondere HasMany e BelongsTo	Sono le due facce della stessa relazione	"Categoria ha molti corpi (hasMany), il corpo appartiene a una categoria (belongsTo)"
Non sapere dove si trova un file	Dimostra mancanza di familiarità col progetto	Usa Ctrl+P e cerca rapidamente